Project Structure

```
educosys-claude/
│
├── context/                        # Context layer
│   ├── indexers/                   # Build indexes
│   ├── retrievers/                 # Query indexes
│   └── memory/                     # Short-term and long-term memory
│
├── agent/                          # Agent reasoning layer
│
├── llm/                            # LLM provider abstraction
│
├── tools/                          # Individual tool functions
│
├── mcp/                            # MCP server integrations
│
├── skills/                         # Higher-level composed capabilities
│
├── safety/                         # Safety layer
│
├── freshness/                      # Freshness layer
│
├── observability/                  # Observability layer
│
└── eval/                           # Evaluation layer
    ├── datasets/                  # Shared golden dataset
    ├── retrieval/                 # Recall@k, MRR, NDCG, Hit Rate
    ├── context/                   # Context precision, context recall
    ├── generation/                # Faithfulness, answer relevancy (RAGAS)
    └── agent/                     # Task success rate, step accuracy
```

Setup

Poetry

Dependency and packaging manager for Python projects. Replaces `pip` + `requirements.txt` + `setup.py` with a single tool.

What it is

Poetry manages three things in one place:

- Your project's dependencies (what packages you need)
- Your virtual environment (isolated Python for this project)
- Building and packaging (turning your project into an installable tool)

All config lives in `pyproject.toml`.

Why you need it

Without Poetry, you'd need:

- `pip install` to install packages
- `requirements.txt` to track what you installed (manually maintained)
- `venv` to create an isolated environment
- `setup.py` to make your project installable

Poetry does all of this automatically and keeps everything in sync.

Key advantages:

- `poetry.lock` pins exact versions of every dependency — your code runs the same on every machine
- Creating a virtualenv is automatic — no manual `python -m venv` step
- Adding a package updates `pyproject.toml` and `poetry.lock` in one command

Install Poetry - https://python-poetry.org/docs

Common commands

```
poetry install          # install all dependencies from poetry.lock
poetry add <package>    # add a new dependency
poetry remove <package> # remove a dependency
poetry run <command>    # run a command inside the project's venv
poetry shell            # activate the venv in your terminal
poetry build            # build the project into a distributable package
```

Key files

`pyproject.toml` - source of truth. Declares dependencies, Python version, scripts, metadata.

`poetry.lock` - auto-generated. Pins exact versions of every package (including transitive dependencies). Commit this to git - it guarantees everyone on the team gets identical installs.

Run educosys_claude in another project

Normally poetry run only works from the project directory. If you want to run educosys_claude from any folder without poetry run, you need to activate the venv first. *poetry env info --path* returns the path to the venv Poetry created for your project. -C <path> tells poetry which project to look at, so you can run this from anywhere. source .../bin/activate puts that venv's binaries on your PATH for the current terminal session. After running it once, educosys_claude works from any directory - just like pipx, but using the editable Poetry install so code changes are reflected immediately.

```
source $(poetry -C /project_path… env info --path)/bin/activate
educosys_claude
```

capstone_project

```
pip install poetry
poetry --version
poetry init
poetry add python-dotenv openai chromadb rich pydantic
```

```
[tool.poetry]
packages = [{include = "educosys_claude"}]

[tool.poetry.scripts]
educosys_claude = "educosys_claude.main:run"
```

capstone_project/pyproject.toml

```
[project]
name = "capstone-project"
version = "0.1.0"
description = ""
authors = [
    {name = "keertipurswani",email = "team@educosys.com"}
]
requires-python = ">=3.12"
dependencies = [
    "python-dotenv (>=1.2.2,<2.0.0)",
    "openai (>=2.43.0,<3.0.0)",
    "chromadb (>=1.5.9,<2.0.0)",
    "rich (>=15.0.0,<16.0.0)",
    "pydantic (>=2.13.4,<3.0.0)"
]

[tool.poetry]
packages = [{include = "educosys_claude"}]

[tool.poetry.scripts]
educosys_claude = "educosys_claude.main:run"

[build-system]
requires = ["poetry-core>=2.0.0,<3.0.0"]
build-backend = "poetry.core.masonry.api"
```

poetry install

.env

```
# Never commit .env to git

# OpenAI
OPENAI_API_KEY=sk-...

# ChromaDB
CHROMA_PERSIST_DIR=.chromadb/
CHROMA_COLLECTION=codebase
EMBEDDING_MODEL=text-embedding-3-small

# Elasticsearch
ELASTICSEARCH_URL=http://localhost:9200
ELASTICSEARCH_INDEX=codebase

# Neo4j
NEO4J_URI=bolt://localhost:7687
NEO4J_USERNAME=neo4j
NEO4J_PASSWORD=your-password

#Langsmith
LANGSMITH_TRACING="true"
LANGSMITH_API_KEY=...
```

.gitignore

```
# Secrets — never commit
.env

# Vector store data
.chromadb/

# Python
__pycache__/
*.pyc
*.pyo

# Poetry
poetry.lock

# Virtual environments
.venv/
venv/
```

Config

educosys_claude/config.yaml

```yaml
embeddings:
 provider: openai
 model: text-embedding-3-small

chromadb:
 persist_dir: .chromadb/
 collection_name: codebase

elasticsearch:
 url: http://localhost:9200
 index_name: codebase
```

educosys_claude/config.py

```python
import yaml
from pathlib import Path

def load_config() -> dict:
    """Load settings from config.yaml at the project root."""
    return yaml.safe_load((Path(__file__).parent / "config.yaml").read_text())

config = load_config()
```

poetry add pyyaml

Logging

educosys_claude/observability/__init__.py
educosys_claude/observability/logger.py

```python
import logging

# Set root logger to WARNING — suppresses noisy third-party library logs (OpenAI, httpcore
    etc.)
logging.basicConfig(
    level=logging.WARNING,
    format="%(asctime)s | %(levelname)s | %(name)s | %(message)s",
)

def get_logger(name: str) -> logging.Logger:
    # Our own loggers run at DEBUG — only third-party libraries are suppressed
    logger = logging.getLogger(name)
    logger.setLevel(logging.DEBUG)
    return logger
```

educosys_claude/main.py

```
from dotenv import load_dotenv
from rich.console import Console
from rich.prompt import Prompt

from educosys_claude.observability.logger import get_logger
# Load .env before anything else — all modules read env vars after this
load_dotenv()
console = Console()
logger = get_logger(__name__)

def run():
  logger.info("Starting Educosys Claude")
  console.print("\n[bold blue]Educosys Claude[/bold blue] — RAG-powered code assistant")
  console.print("Type [bold]'/exit'[/bold] to quit\n")

  while True:
      user_input = Prompt.ask("[bold green]>[/bold green]")
      if not user_input.strip():
          continue
      if user_input.lower() in ("/exit", "/quit"):
          logger.info("Shutting down")
          console.print("[dim]Goodbye![/dim]")
          break
      elif user_input.startswith("/ask "):
          question = user_input.removeprefix("/ask ").strip()
          logger.info(f"Ask command received: {question}")
          console.print(f"[dim]Searching for: {question}...[/dim]")
          # TODO: call retriever + LLM
      else:
          logger.warning(f"Unknown command received: {user_input}")
          console.print("[yellow]Unknown command. Try:[/yellow]")
          console.print("  [bold]index <path>[/bold]  — index a codebase")
          console.print("  [bold]ask <question>[/bold] — ask a question")

if __name__ == "__main__":
  run()
```

Run ->

poetry run educosys_claude

LLM Providers

config.yaml

```yaml
llm:
 provider: openai        # openai | anthropic
 model: gpt-4o
```

llm/factory.py

```python
from educosys_claude.config import config
from educosys_claude.observability.logger import get_logger

logger = get_logger(__name__)

def get_llm():
  """Return the right LangChain LLM based on config."""
  provider = config["llm"]["provider"]
  model = config["llm"]["model"]
  logger.info(f"Using LLM provider: {provider}, model: {model}")

  if provider == "anthropic":
      from langchain_anthropic import ChatAnthropic
      return ChatAnthropic(model=model)

  from langchain_openai import ChatOpenAI
  return ChatOpenAI(model=model)

def get_embedder():
  """Return the right LangChain embedder based on config."""
  provider = config["embeddings"]["provider"]
  model = config["embeddings"]["model"]
  logger.info(f"Using embeddings provider: {provider}, model: {model}")
  if provider == "huggingface":
      from langchain_huggingface import HuggingFaceEmbeddings
      return HuggingFaceEmbeddings(model_name=model)
  from langchain_openai import OpenAIEmbeddings
  return OpenAIEmbeddings(model=model)
```

poetry add langchain langchain-openai langchain-anthropic langchain-huggingface --python ">=3.12,<4.0.0"

Pipx

https://pipx.pypa.io/stable/

Install pipx - https://pipx.pypa.io/stable/how-to/install-pipx/

Install your tool

```
cd /path/to/your/project
pipx install .
```

Run once. After this, the command is globally available.

After code changes — reinstall

```
pipx reinstall <tool-name>
```

Required every time you change code or dependencies. pipx copies files at install time - edits to source don't auto-reflect.

Key requirements for project

- `pyproject.toml` must declare a `[tool.poetry.scripts]` entry point
- Non-Python files (`.env`, `config.yaml`) must be listed under `packages` so they get copied
- File paths in code must use `Path(__file__).parent / "file"` — not relative paths like `Path("file")`

Useful commands

- pipx install .
- pipx list # see installed tools
- pipx uninstall <tool-name> # remove a tool
- pipx inject <tool> <package> # add a package to a tool's venv

Semantic RAG - Chroma

Code Parsing

context/indexers/code_parser.py

```python
from pathlib import Path
from dataclasses import dataclass

from tree_sitter import Language, Parser
from tree_sitter_languages import get_language, get_parser

from educosys_claude.observability.logger import get_logger

logger = get_logger(__name__)

# Maps file extension → tree-sitter language name.
# tree-sitter-languages bundles all these grammars, no extra installs needed.
EXTENSION_TO_LANGUAGE = {
  ".py": "python",
  ".js": "javascript",
  ".jsx": "javascript",
  ".ts": "typescript",
  ".tsx": "tsx",
  ".java": "java",
  ".go": "go",
  ".rs": "rust",
  ".cpp": "cpp",
  ".c": "c",
  ".cs": "c_sharp",
  ".rb": "ruby",
  ".php": "php",
  ".swift": "swift",
  ".kt": "kotlin",
  ".sh": "bash",
}


# Text/config files have no meaningful AST — we chunk them by line count instead.
TEXT_EXTENSIONS = {".md", ".txt", ".yaml", ".yml", ".json", ".toml"}
ALL_EXTENSIONS = set(EXTENSION_TO_LANGUAGE.keys()) | TEXT_EXTENSIONS

# Sliding window settings for text files and AST fallback.
# CHUNK_OVERLAP ensures context isn't lost at chunk boundaries.
CHUNK_SIZE = 50
CHUNK_OVERLAP = 10

# Tree-sitter node type names that correspond to a named, indexable block.
# These are consistent across languages — tree-sitter uses the same names where possible.
BLOCK_NODE_TYPES = {
  "function_definition", "function_declaration", "method_definition",
  "arrow_function", "class_definition", "class_declaration",
  "method_declaration", "constructor_declaration", "interface_declaration",
  "function_item",    # Rust uses this instead of function_definition
  "func_declaration", # Go
}

@dataclass
class ParsedChunk:
  name: str
  type: str        # "function" | "class" | "block"
  content: str
  source: str      # absolute path to the file
  start_line: int
```

```
    end_line: int

def parse_file(filepath: str) -> list[ParsedChunk]:
    """Entry point — routes to AST parsing or sliding window based on file type."""
    ext = Path(filepath).suffix.lower()

    if ext in TEXT_EXTENSIONS:
        lines = Path(filepath).read_text(encoding="utf-8", errors="ignore").splitlines()
        return _sliding_window(lines, filepath)

    language_name = EXTENSION_TO_LANGUAGE.get(ext)
    if not language_name:
        raise ValueError(f"Unsupported file type: {ext}")

    source = Path(filepath).read_text(encoding="utf-8", errors="ignore")
    return _parse_with_treesitter(source, filepath, language_name)

def _parse_with_treesitter(source: str, filepath: str, language_name: str) ->
     list[ParsedChunk]:
    """Parse source with the appropriate tree-sitter grammar and extract named blocks."""
    logger.info(f"Parsing {language_name} file: {filepath}")
    parser = get_parser(language_name)
    tree = parser.parse(source.encode())
    lines = source.splitlines()

    chunks = []
    _walk(tree.root_node, source, filepath, chunks, depth=0)

    # If the AST yielded nothing (e.g. a file with only imports), fall back to line chunks
    if not chunks:
        logger.warning(f"No AST blocks found in {filepath}, falling back to sliding window")
        return _sliding_window(lines, filepath)

    logger.info(f"Parsed {len(chunks)} chunks from {filepath}")
    return chunks

def _walk(node, source: str, filepath: str, chunks: list, depth: int):
    """
    Recursively walk the AST. When a named block node is found, record it and stop
    descending — this keeps chunks at the top level and avoids duplicating nested functions.
    """
    if node.type in BLOCK_NODE_TYPES:
        name = _extract_name(node, source)
        content = source[node.start_byte:node.end_byte]
        chunk_type = "class" if "class" in node.type else "function"
        chunks.append(ParsedChunk(
            name=name,
            type=chunk_type,
            content=content,
            source=filepath,
            start_line=node.start_point[0] + 1,
            end_line=node.end_point[0] + 1,
        ))
        logger.debug(f"  Found {chunk_type} '{name}' (lines
     {node.start_point[0]+1}-{node.end_point[0]+1})")
        return  # stop here — don't index nested functions/classes as separate chunks

    for child in node.children:
        _walk(child, source, filepath, chunks, depth + 1)

def _extract_name(node, source: str) -> str:
    """Find the identifier child of a block node and return its text as the chunk name."""
    for child in node.children:
        if child.type in ("identifier", "name", "property_identifier"):
            return source[child.start_byte:child.end_byte]
    return node.type  # fallback to node type if no name found

def _sliding_window(lines: list[str], filepath: str) -> list[ParsedChunk]:
```

```
    """
    Split lines into overlapping fixed-size chunks.
    Used for text files and as a fallback when AST parsing finds nothing.
    """
    if not lines:
        raise ValueError(f"Empty file: {filepath}")

    chunks = []
    step = CHUNK_SIZE - CHUNK_OVERLAP
    for i, start in enumerate(range(0, len(lines), step)):
        end = min(start + CHUNK_SIZE, len(lines))
        text = "\n".join(lines[start:end]).strip()
        if text:
            chunks.append(ParsedChunk(
                name=f"chunk_{i}", type="block", content=text,
                source=filepath, start_line=start + 1, end_line=end,
            ))
        if end == len(lines):
            break

    logger.info(f"Parsed {len(chunks)} chunks from {filepath}")
    return chunks

def get_source_files(repo_path: str, skip_dirs: list[str] = None) -> list[str]:
    """Recursively find all indexable source files under repo_path, skipping build/env
     dirs."""
    skip = set(skip_dirs or [".venv", "venv", "__pycache__", ".git", "node_modules", "dist",
     "build"])
    files = [
        str(path) for path in Path(repo_path).rglob("*")
        if path.suffix.lower() in ALL_EXTENSIONS
        and not any(part in skip for part in path.parts)
    ]
    logger.info(f"Found {len(files)} source files in {repo_path}")
    return files
```

poetry add tree-sitter==0.21.3 tree-sitter-languages==1.10.2

Chroma Indexing

semantic_chroma.py

```
import chromadb

from educosys_claude.config import config
from educosys_claude.context.indexers.code_parser import parse_file, get_source_files
from educosys_claude.llm.factory import get_embedder
from educosys_claude.observability.logger import get_logger

logger = get_logger(__name__)

def index_codebase(repo_path: str) -> chromadb.Collection:
 """
 Parse all Python files in repo_path, embed each chunk and store in ChromaDB.
 Returns the ChromaDB collection for use by the retriever.
 """
 embedder = get_embedder()
 chroma_client = chromadb.PersistentClient(path=config["chromadb"]["persist_dir"])
 collection =
   chroma_client.get_or_create_collection(name=config["chromadb"]["collection_name"])

 logger.info(f"Starting semantic indexing of {repo_path}")

 files = get_source_files(repo_path)


 for filepath in files:
     try:
         chunks = parse_file(filepath)
     except (SyntaxError, ValueError) as e:
         logger.error(f"Skipping {filepath}: {e}")
         continue

     for chunk in chunks:
         embedding = embedder.embed_query(chunk.content)

         # Unique ID for each chunk — source + name + line prevents duplicates on re-index
         doc_id = f"{chunk.source}::{chunk.name}::{chunk.start_line}"

         collection.upsert(
             ids=[doc_id],
             embeddings=[embedding],
             documents=[chunk.content],
             metadatas=[{
                 "source": chunk.source,
                 "name": chunk.name,
                 "type": chunk.type,
                 "start_line": chunk.start_line,
                 "end_line": chunk.end_line,
             }]
         )
         logger.debug(f"  Indexed {chunk.type} '{chunk.name}' from {filepath}")

 logger.info(f"Semantic indexing complete. Total chunks: {collection.count()}")
 return collection
```

main.py

```
import os
from pathlib import Path
from dotenv import load_dotenv
from rich.console import Console
from rich.prompt import Prompt
import chromadb

from educosys_claude.config import config
from educosys_claude.context.indexers.semantic_chroma import index_codebase
from educosys_claude.llm.factory import get_llm, get_embedder
from educosys_claude.observability.logger import get_logger

# Load .env before anything else
load_dotenv()
console = Console()
logger = get_logger(__name__)

def get_or_create_index() -> chromadb.Collection:
 """Load existing index or create a new one by indexing current directory."""
 chroma_client = chromadb.PersistentClient(path=config["chromadb"]["persist_dir"])
 collection =
    chroma_client.get_or_create_collection(name=config["chromadb"]["collection_name"])

 if collection.count() > 0:
     logger.info(f"Loaded existing index with {collection.count()} chunks")
     console.print(f"[dim]Loaded existing index — {collection.count()} chunks[/dim]")
     return collection

 # Auto-index the current working directory
 repo_path = str(Path.cwd())
 logger.info(f"Indexing repo: {repo_path}")
 console.print(f"[dim]Indexing {repo_path}...[/dim]")
 return index_codebase(repo_path)

def show_semantic_index(collection: chromadb.Collection) -> None:
 """Display all documents stored in the ChromaDB collection."""
 results = collection.get(include=["documents", "metadatas", "embeddings"])
 console.print(f"\n[bold]Semantic Index — {collection.count()} chunks[/bold]\n")

 for i, (doc, meta, emb) in enumerate(zip(
     results["documents"],
     results["metadatas"],
     results["embeddings"]
 )):
     console.print(f"[bold cyan]── Chunk {i+1} ──────────────────────────[/bold cyan]")
     console.print(f"  File     : {meta['source']}")
     console.print(f"  Name     : {meta['name']} ({meta['type']})")
     console.print(f"  Lines    : {meta['start_line']} - {meta['end_line']}")
     console.print(f"  Embedding: [{', '.join(f'{v:.4f}' for v in emb[:5])}...] ({len(emb)}
    dims)")
     console.print(f"  Code     :\n[dim]{doc[:300]}[/dim]\n")


def initialize():
 """Bootstrap LLM, embedder, and index before the REPL starts."""
 llm = get_llm()
 embedder = get_embedder()
 console.print(f"[dim]LLM: {config['llm']['provider']} / {config['llm']['model']}[/dim]")
 console.print(f"[dim]Embedder: {config['embeddings']['provider']} /
    {config['embeddings']['model']}[/dim]")

 collection = get_or_create_index()
 console.print(f"[green]✓ Ready — {collection.count()} chunks indexed[/green]\n")
 return llm, embedder, collection
```

```
def run():
 logger.info("Starting Educosys Claude")
 console.print("\n[bold blue]Educosys Claude[/bold blue] — RAG-powered code assistant")

 llm, embedder, collection = initialize()
 console.print("Type [bold]'/exit'[/bold] to quit\n")

 while True:
     user_input = Prompt.ask("[bold green]>[/bold green]")

     if not user_input.strip():
         continue
     if user_input.lower() in ("/exit", "/quit"):
         logger.info("Shutting down")
         console.print("[dim]Goodbye![/dim]")
         break
     elif user_input.startswith("/ask "):
         question = user_input.removeprefix("/ask ").strip()
         logger.info(f"Ask command received: {question}")
         console.print(f"[dim]Searching for: {question}...[/dim]")
         # TODO: call retriever + LLM
     elif user_input == "/show_semantic_index":
         logger.info("Showing semantic index")
         show_semantic_index(collection)
     else:
         logger.warning(f"Unknown command received: {user_input}")
         console.print("[yellow]Unknown command. Try:[/yellow]")
         console.print("  [bold]ask <question>[/bold]       — ask a question about the
    codebase")
         console.print("  [bold]show_semantic_index[/bold]  — show all chunks in the
    semantic index")

if __name__ == "__main__":
 run()
```

Chroma Retriever

retrievers/semantic_chroma.py

```
import chromadb
from educosys_claude.config import config
from educosys_claude.llm.factory import get_embedder
from educosys_claude.observability.logger import get_logger

logger = get_logger(__name__)

def retrieve(query: str, k: int = 5) -> list[dict]:
  """
  Embed the query and find the k most similar chunks in ChromaDB.
  Returns a list of results with content and metadata.
  """
  chroma_client = chromadb.PersistentClient(path=config["chromadb"]["persist_dir"])
  collection =
    chroma_client.get_or_create_collection(name=config["chromadb"]["collection_name"])

  logger.info(f"Retrieving top {k} chunks for query: {query}")

  # Use the same embedder as the indexer — model must match or results will be wrong
  embedder = get_embedder()
  query_embedding = embedder.embed_query(query)

  results = collection.query(
      query_embeddings=[query_embedding],
      n_results=k,
      include=["documents", "metadatas", "distances"]
  )
  chunks = []
  for doc, meta, dist in zip(
      results["documents"][0],
      results["metadatas"][0],
      results["distances"][0]
  ):
      chunks.append({
          "content": doc,
          "source": meta["source"],
          "name": meta["name"],
          "type": meta["type"],
          "start_line": meta["start_line"],
          "end_line": meta["end_line"],
          "distance": dist,   # lower = more similar
      })
      logger.debug(f"  Retrieved {meta['type']} '{meta['name']}' from {meta['source']}
    (distance: {dist:.4f})")

  logger.info(f"Retrieved {len(chunks)} chunks")
  return chunks
```

agent/tools.py

```
from langchain.tools import tool

from educosys_claude.context.retrievers.semantic_chroma import retrieve
from educosys_claude.observability.logger import get_logger

logger = get_logger(__name__)

@tool
def search_codebase(query: str) -> str:
    """
```

```
    Search the codebase for relevant classes, functions or logic.
    Use this tool whenever you need to find code related to a question.
    """
    logger.info(f"Tool called: search_codebase with query: {query}")
    chunks = retrieve(query, k=5)

    if not chunks:
        return "No relevant code found."

    # Format chunks into a readable string for the LLM
    results = []
    for chunk in chunks:
        results.append(
            f"File: {chunk['source']} (lines {chunk['start_line']}-{chunk['end_line']})\n"
            f"Type: {chunk['type']} — {chunk['name']}\n"
            f"Code:\n{chunk['content']}\n"
        )
    return "\n---\n".join(results)
```

Agent

agent/factory.py

```
from langchain.agents import create_agent
from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder

from educosys_claude.llm.factory import get_llm
from educosys_claude.agent.tools import search_codebase
from educosys_claude.observability.logger import get_logger

logger = get_logger(__name__)

SYSTEM_PROMPT = """You are a senior software engineer with deep knowledge of the codebase.
Always use the search_codebase tool before answering any question.
Reference specific file names, function names and line numbers in your answers.
If you cannot find the answer in the codebase, say so explicitly."""

def build_agent():
  """Create and return a LangChain agent."""
  llm = get_llm()
  tools = [search_codebase]
  logger.info("Creating agent")
  return create_agent(
      llm,
      tools=tools,
      system_prompt=SYSTEM_PROMPT,
  )
```

agent/orchestrator.py

```
from educosys_claude.agent.factory import build_agent
from educosys_claude.observability.logger import get_logger

logger = get_logger(__name__)

def handle_query(question: str) -> str:
    """Entry point for all user queries - builds the agent and runs it."""
    logger.info(f"Handling query: {question}")
    agent = build_agent()
    response = agent.invoke({"messages": [{"role": "user", "content": question}]})
    return response["messages"][-1].content
    return response["output"]
```

main.py

```
from educosys_claude.agent.orchestrator import handle_query
```

```
elif user_input.startswith("/ask "):
        question = user_input.removeprefix("/ask ").strip()
        logger.info(f"Ask command received: {question}")
        console.print(f"[dim]Searching for: {question}...[/dim]")
        response = handle_query(question)
        console.print(response)
```

Semantic RAG - Qdrant

https://docs.langchain.com/oss/python/integrations/vectorstores/qdrant
https://qdrant.tech

poetry add "langchain-qdrant>=1.1.0,<2.0.0" --python ">=3.12,<4.0.0"

.env

```
#Qdrant
QDRANT_API_KEY=...
QDRANT_URL=https://....aws.cloud.qdrant.io
```

Indexing

config.yaml

```
vector_store:
 provider: qdrant

qdrant:
 collection_name: codebase
```

indexers/factory.py

```
from educosys_claude.config import config
from educosys_claude.observability.logger import get_logger

logger = get_logger(__name__)

def get_indexer():
   """Return the right index_codebase function based on vector_store in config."""
   vector_store = config["vector_store"]["provider"]
   logger.info(f"Using vector store: {vector_store}")

   if vector_store == "qdrant":
       from .semantic_qdrant import index_codebase
   else:
       from .semantic_chroma import index_codebase
   return index_codebase

def get_index_inspector():
   """Return the right show_index function based on vector_store in config."""
   vector_store = config["vector_store"]["provider"]

   if vector_store == "qdrant":
       from .semantic_qdrant import show_index
   else:
       from .semantic_chroma import show_index
   return show_index
```

indexers/semantic_qdrant.py

```
import os
from langchain_qdrant import QdrantVectorStore
from langchain_core.documents import Document
from qdrant_client import QdrantClient

from educosys_claude.config import config
from educosys_claude.context.indexers.code_parser import parse_file, get_source_files
from educosys_claude.llm.factory import get_embedder
from educosys_claude.observability.logger import get_logger

logger = get_logger(__name__)


def index_codebase(repo_path: str) -> QdrantVectorStore:
    """
    Parse all source files in repo_path, embed each chunk and store in Qdrant.
    Returns the QdrantVectorStore. Skips indexing if collection already has data.
    """
    embedder = get_embedder()
    collection_name = config["qdrant"]["collection_name"]
    url = os.getenv("QDRANT_URL")
    api_key = os.getenv("QDRANT_API_KEY")

    # Check if collection already has data
    client = QdrantClient(url=url, api_key=api_key)
    existing = [c.name for c in client.get_collections().collections]
    if collection_name in existing:
        info = client.get_collection(collection_name)
        if info.points_count > 0:
            logger.info(f"Loaded existing index with {info.points_count} chunks")
            return QdrantVectorStore.from_existing_collection(
                embedding=embedder,
                url=url,
                api_key=api_key,
                collection_name=collection_name,
            )

    logger.info(f"Starting semantic indexing of {repo_path}")
    files = get_source_files(repo_path)
    docs = []

    for filepath in files:
        try:
            chunks = parse_file(filepath)
        except (SyntaxError, ValueError) as e:
            logger.error(f"Skipping {filepath}: {e}")
            continue

        for chunk in chunks:
            docs.append(Document(
                page_content=chunk.content,
                metadata={
                    "source": chunk.source,
                    "name": chunk.name,
                    "type": chunk.type,
                    "start_line": chunk.start_line,
                    "end_line": chunk.end_line,
                }
            ))
            logger.debug(f"  Indexed {chunk.type} '{chunk.name}' from {filepath}")

    vector_store = QdrantVectorStore.from_documents(
        docs, embedder,
        url=url,
        api_key=api_key,
```

```
        collection_name=collection_name,
        batch_size=50,
    )

    logger.info(f"Semantic indexing complete. Total chunks: {len(docs)}")
    return vector_store


def show_index(vector_store: QdrantVectorStore) -> None:
    """Display all documents stored in the Qdrant collection."""
    from rich.console import Console
    console = Console()
    client = vector_store.client
    collection_name = config["qdrant"]["collection_name"]
    results = client.scroll(collection_name=collection_name, with_payload=True,
    with_vectors=True, limit=1000)
    points = results[0]
    console.print(f"\n[bold]Semantic Index — {len(points)} chunks[/bold]\n")

    for i, point in enumerate(points):
        payload = point.payload
        emb = point.vector
        console.print(f"[bold cyan]── Chunk {i+1} ──────────────────────────[/bold cyan]")
        console.print(f"  File      : {payload['source']}")
        console.print(f"  Name      : {payload['name']} ({payload['type']})")
        console.print(f"  Lines     : {payload['start_line']} - {payload['end_line']}")
        console.print(f"  Embedding: [{', '.join(f'{v:.4f}' for v in emb[:5])}...]
    ({len(emb)} dims)")
        console.print(f"  Code      :\n[dim]{payload.get('page_content', '')[:300]}[/dim]\n")
```

indexers/semantic_chroma.py

```
import chromadb

from educosys_claude.config import config
from educosys_claude.context.indexers.code_parser import parse_file, get_source_files
from educosys_claude.llm.factory import get_embedder
from educosys_claude.observability.logger import get_logger

logger = get_logger(__name__)

def index_codebase(repo_path: str) -> chromadb.Collection:
    """
    Parse all source files in repo_path, embed each chunk and store in ChromaDB.
    Returns the ChromaDB collection. Skips indexing if collection already has data.
    """
    embedder = get_embedder()
    chroma_client = chromadb.PersistentClient(path=config["chromadb"]["persist_dir"])
    collection =
    chroma_client.get_or_create_collection(name=config["chromadb"]["collection_name"])

    if collection.count() > 0:
        logger.info(f"Loaded existing index with {collection.count()} chunks")
        return collection

    logger.info(f"Starting semantic indexing of {repo_path}")
    files = get_source_files(repo_path)

    for filepath in files:
        try:
            chunks = parse_file(filepath)
        except (SyntaxError, ValueError) as e:
            logger.error(f"Skipping {filepath}: {e}")
            continue

        for chunk in chunks:
            embedding = embedder.embed_query(chunk.content)
```

```
            doc_id = f"{chunk.source}::{chunk.name}::{chunk.start_line}"
            collection.upsert(
                ids=[doc_id],
                embeddings=[embedding],
                documents=[chunk.content],
                metadatas=[{
                    "source": chunk.source,
                    "name": chunk.name,
                    "type": chunk.type,
                    "start_line": chunk.start_line,
                    "end_line": chunk.end_line,
                }]
            )
            logger.debug(f"  Indexed {chunk.type} '{chunk.name}' from {filepath}")

    logger.info(f"Semantic indexing complete. Total chunks: {collection.count()}")
    return collection


def show_index(collection: chromadb.Collection) -> None:
    """Display all documents stored in the ChromaDB collection."""
    from rich.console import Console
    console = Console()
    results = collection.get(include=["documents", "metadatas", "embeddings"])
    console.print(f"\n[bold]Semantic Index — {collection.count()} chunks[/bold]\n")

    for i, (doc, meta, emb) in enumerate(zip(
        results["documents"],
        results["metadatas"],
        results["embeddings"]
    )):
        console.print(f"[bold cyan]── Chunk {i+1} ──────────────────────────[/bold cyan]")
        console.print(f"  File     : {meta['source']}")
        console.print(f"  Name     : {meta['name']} ({meta['type']})")
        console.print(f"  Lines    : {meta['start_line']} - {meta['end_line']}")
        console.print(f"  Embedding: [{', '.join(f'{v:.4f}' for v in emb[:5])}...]
    ({len(emb)} dims)")
        console.print(f"  Code     :\n[dim]{doc[:300]}[/dim]\n")
```

main.py

```
import os
from pathlib import Path
from dotenv import load_dotenv
from rich.console import Console
from rich.prompt import Prompt

from educosys_claude.config import config
from educosys_claude.context.indexers.factory import get_indexer, get_index_inspector
from educosys_claude.llm.factory import get_llm, get_embedder
from educosys_claude.agent.orchestrator import handle_query
from educosys_claude.observability.logger import get_logger

# Load .env before anything else
load_dotenv(Path(__file__).parent.parent / ".env")

console = Console()
logger = get_logger(__name__)

def get_or_create_index():
    repo_path = str(Path.cwd())
    logger.info(f"Checking index for: {repo_path}")
    console.print(f"[dim]Checking index for {repo_path}...[/dim]")
    return get_indexer()(repo_path)

def initialize():
    """Bootstrap LLM, embedder, and index before the REPL starts."""
```

```
    llm = get_llm()
    embedder = get_embedder()
    console.print(f"[dim]LLM: {config['llm']['provider']} / {config['llm']['model']}[/dim]")
    console.print(f"[dim]Embedder: {config['embeddings']['provider']} /
    {config['embeddings']['model']}[/dim]")

    index = get_or_create_index()
    console.print(f"[green]✓ Ready[/green]\n")
    return llm, embedder, index

def run():
    logger.info("Starting Educosys Claude")
    console.print("\n[bold blue]Educosys Claude[/bold blue] — RAG-powered code assistant")

    llm, embedder, index = initialize()
    console.print("Type [bold]'/exit'[/bold] to quit\n")

    while True:
        user_input = Prompt.ask("[bold green]>[/bold green]")

        if not user_input.strip():
            continue
        if user_input.lower() in ("/exit", "/quit"):
            logger.info("Shutting down")
            console.print("[dim]Goodbye![/dim]")
            break
        elif user_input.startswith("/ask "):
            question = user_input.removeprefix("/ask ").strip()
            logger.info(f"Ask command received: {question}")
            console.print(f"[dim]Searching for: {question}...[/dim]")
            response = handle_query(question)
            console.print(response)
        elif user_input == "/show_semantic_index":
            logger.info("Showing semantic index")
            get_index_inspector()(index)
        else:
            logger.warning(f"Unknown command received: {user_input}")
            console.print("[yellow]Unknown command. Try:[/yellow]")
            console.print("  [bold]/ask <question>[/bold]       — ask a question about the
    codebase")
            console.print("  [bold]/show_semantic_index[/bold]  — show all chunks in the
    semantic index")

if __name__ == "__main__":
    run()
```

Retrieval

retrievers/factory.py

```
from educosys_claude.config import config
from educosys_claude.observability.logger import get_logger

logger = get_logger(__name__)

def get_retriever():
   """Return the right retrieve function based on vector_store in config."""
   vector_store = config["vector_store"]["provider"]
   logger.info(f"Using vector store for retrieval: {vector_store}")

   if vector_store == "qdrant":
       from .semantic_qdrant import retrieve
   else:
       from .semantic_chroma import retrieve

   return retrieve
```

retrievers/semantic_qdrant.py

```
import os
from langchain_qdrant import QdrantVectorStore

from educosys_claude.config import config
from educosys_claude.llm.factory import get_embedder
from educosys_claude.observability.logger import get_logger

logger = get_logger(__name__)

def retrieve(query: str, k: int = 5) -> list[dict]:
   """
   Embed the query and find the k most similar chunks in Qdrant.
   Returns a list of results with content and metadata.
   """
   embedder = get_embedder()

   vector_store = QdrantVectorStore.from_existing_collection(
       embedding=embedder,
       url=os.getenv("QDRANT_URL"),
       api_key=os.getenv("QDRANT_API_KEY"),
       collection_name=config["qdrant"]["collection_name"],
   )

   logger.info(f"Retrieving top {k} chunks for query: {query}")
   results = vector_store.similarity_search_with_score(query, k=k)

   chunks = []
   for doc, score in results:
       meta = doc.metadata
       chunks.append({
           "content": doc.page_content,
           "source": meta["source"],
           "name": meta["name"],
           "type": meta["type"],
           "start_line": meta["start_line"],
           "end_line": meta["end_line"],
           "distance": score,
       })
       logger.debug(f"  Retrieved {meta['type']} '{meta['name']}' from {meta['source']}
   (score: {score:.4f})")

   logger.info(f"Retrieved {len(chunks)} chunks")
   return chunks
```

retrievers/semantic_chroma.py - unchanged

agent/tools.py

```python
from langchain.tools import tool

from educosys_claude.context.retrievers.factory import get_retriever
from educosys_claude.observability.logger import get_logger

logger = get_logger(__name__)

@tool
def search_codebase(query: str) -> str:
    """
    Search the codebase for relevant classes, functions or logic.
    Use this tool whenever you need to find code related to a question.
    """
    logger.info(f"Tool called: search_codebase with query: {query}")
    retrieve = get_retriever()
    chunks = retrieve(query, k=5)
    if not chunks:
        return "No relevant code found."

    results = []
    for chunk in chunks:
        results.append(
            f"File: {chunk['source']} (lines {chunk['start_line']}-{chunk['end_line']})\n"
            f"Type: {chunk['type']} — {chunk['name']}\n"
            f"Code:\n{chunk['content']}\n"
        )
    return "\n---\n".join(results)
```

Hybrid RAG with Qdrant

poetry add fastembed

Indexing

indexers/hybrid_qdrant.py

```
import os
from langchain_qdrant import QdrantVectorStore, RetrievalMode, FastEmbedSparse
from langchain_core.documents import Document
from qdrant_client import QdrantClient

from educosys_claude.config import config
from educosys_claude.context.indexers.code_parser import parse_file, get_source_files
from educosys_claude.llm.factory import get_embedder
from educosys_claude.observability.logger import get_logger

logger = get_logger(__name__)

RETRIEVAL_MODE_MAP = {
    "dense": RetrievalMode.DENSE,
    "sparse": RetrievalMode.SPARSE,
    "hybrid": RetrievalMode.HYBRID,
}


def _get_retrieval_mode() -> RetrievalMode:
    mode = config["vector_store"].get("retrieval_mode", "hybrid")
    return RETRIEVAL_MODE_MAP.get(mode, RetrievalMode.HYBRID)


def index_codebase(repo_path: str) -> QdrantVectorStore:
    """
    Parse all source files, embed with dense + sparse vectors and store in Qdrant.
    Supports dense, sparse, and hybrid retrieval modes via config.
    Skips indexing if collection already has data.
    """
    embedder = get_embedder()
    collection_name = config["qdrant"]["collection_name"]
    url = os.getenv("QDRANT_URL")
    api_key = os.getenv("QDRANT_API_KEY")
    retrieval_mode = _get_retrieval_mode()

    # Check if collection already has data
    client = QdrantClient(url=url, api_key=api_key)
    existing = [c.name for c in client.get_collections().collections]
    if collection_name in existing:
        info = client.get_collection(collection_name)
        if info.points_count > 0:
            logger.info(f"Loaded existing index with {info.points_count} chunks")
            return QdrantVectorStore.from_existing_collection(
                embedding=embedder,
                sparse_embedding=FastEmbedSparse(model_name="Qdrant/bm25"),
                retrieval_mode=retrieval_mode,
                url=url,
                api_key=api_key,
                collection_name=collection_name,
            )

    logger.info(f"Starting hybrid indexing of {repo_path}")
    files = get_source_files(repo_path)
```

```
    docs = []

    for filepath in files:
        try:
            chunks = parse_file(filepath)
        except (SyntaxError, ValueError) as e:
            logger.error(f"Skipping {filepath}: {e}")
            continue

        for chunk in chunks:
            docs.append(Document(
                page_content=chunk.content,
                metadata={
                    "source": chunk.source,
                    "name": chunk.name,
                    "type": chunk.type,
                    "start_line": chunk.start_line,
                    "end_line": chunk.end_line,
                }
            ))
            logger.debug(f"  Indexed {chunk.type} '{chunk.name}' from {filepath}")

    vector_store = QdrantVectorStore.from_documents(
        docs, embedder,
        sparse_embedding=FastEmbedSparse(model_name="Qdrant/bm25"),
        retrieval_mode=retrieval_mode,
        url=url,
        api_key=api_key,
        collection_name=collection_name,
        batch_size=50,
    )

    logger.info(f"Hybrid indexing complete. Total chunks: {len(docs)}")
    return vector_store

def show_index(vector_store: QdrantVectorStore) -> None:
    from rich.console import Console
    console = Console()
    client = vector_store.client
    collection_name = config["qdrant"]["collection_name"]
    results = client.scroll(collection_name=collection_name, with_payload=True,
    with_vectors=False, limit=1000)
    points = results[0]
    console.print(f"\n[bold]Hybrid Index — {len(points)} chunks[/bold]\n")

    for i, point in enumerate(points):
        content = point.payload.get("page_content", "")
        meta = point.payload.get("metadata", {})
        console.print(f"[bold cyan]── Chunk {i+1} ──────────────────────────[/bold cyan]")
        console.print(f"  File     : {meta.get('source', 'unknown')}")
        console.print(f"  Name     : {meta.get('name', 'unknown')} ({meta.get('type',
    'unknown')})")
        console.print(f"  Lines    : {meta.get('start_line')} - {meta.get('end_line')}")
        console.print(f"  Code     :\n[dim]{content[:300]}[/dim]\n")
```

Retrieval

retrievers/hybrid_qdrant.py

```python
import os
from langchain_qdrant import QdrantVectorStore, RetrievalMode, FastEmbedSparse

from educosys_claude.config import config
from educosys_claude.llm.factory import get_embedder
from educosys_claude.observability.logger import get_logger

logger = get_logger(__name__)

RETRIEVAL_MODE_MAP = {
    "dense": RetrievalMode.DENSE,
    "sparse": RetrievalMode.SPARSE,
    "hybrid": RetrievalMode.HYBRID,
}

def retrieve(query: str, k: int = 5) -> list[dict]:
    """
    Retrieve top-k chunks using dense, sparse, or hybrid mode — controlled by config.
    """
    embedder = get_embedder()
    collection_name = config["qdrant"]["collection_name"]
    mode = config["vector_store"].get("retrieval_mode", "hybrid")
    retrieval_mode = RETRIEVAL_MODE_MAP.get(mode, RetrievalMode.HYBRID)

    vector_store = QdrantVectorStore.from_existing_collection(
        embedding=embedder,
        sparse_embedding=FastEmbedSparse(model_name="Qdrant/bm25"),
        retrieval_mode=retrieval_mode,
        url=os.getenv("QDRANT_URL"),
        api_key=os.getenv("QDRANT_API_KEY"),
        collection_name=collection_name,
    )

    logger.info(f"Retrieving top {k} chunks — mode: {mode} — query: {query}")
    results = vector_store.similarity_search_with_score(query, k=k)

    chunks = []
    for doc, score in results:
        meta = doc.metadata
        chunks.append({
            "content": doc.page_content,
            "source": meta["source"],
            "name": meta["name"],
            "type": meta["type"],
            "start_line": meta["start_line"],
            "end_line": meta["end_line"],
            "distance": score,
        })
        logger.debug(f"  Retrieved {meta['type']} '{meta['name']}' from {meta['source']}
    (score: {score:.4f})")

    logger.info(f"Retrieved {len(chunks)} chunks")
    return chunks
```

config.yaml

```yaml
rag:
 mode: hybrid           # semantic | hybrid

vector_store:
 provider: qdrant
 retrieval_mode: hybrid  # dense | sparse | hybrid
```

indexers/factory.py

```python
from educosys_claude.config import config
from educosys_claude.observability.logger import get_logger

logger = get_logger(__name__)

def get_indexer():
   mode = config["rag"]["mode"]
   provider = config["vector_store"]["provider"]

   if mode == "hybrid" and provider == "qdrant":
       from .hybrid_qdrant import index_codebase
   elif provider == "qdrant":
       from .semantic_qdrant import index_codebase
   else:
       from .semantic_chroma import index_codebase

   return index_codebase

def get_index_inspector():
   """Return the right show_index function based on rag mode and vector_store in config."""
   mode = config["rag"]["mode"]
   provider = config["vector_store"]["provider"]

   if mode == "hybrid" and provider == "qdrant":
       from .hybrid_qdrant import show_index
   elif provider == "qdrant":
       from .semantic_qdrant import show_index
   else:
       from .semantic_chroma import show_index

   return show_index
```

retrievers/factory.py

```python
from educosys_claude.config import config
from educosys_claude.observability.logger import get_logger

logger = get_logger(__name__)


def get_retriever():
   mode = config["rag"]["mode"]
   provider = config["vector_store"]["provider"]

   if mode == "hybrid" and provider == "qdrant":
       from .hybrid_qdrant import retrieve
   elif provider == "qdrant":
       from .semantic_qdrant import retrieve
   else:
       from .semantic_chroma import retrieve

   return retrieve
```

Memory

STM

poetry add langgraph-checkpoint-sqlite

memory/session.py

```
import uuid
from pathlib import Path

from educosys_claude.config import config
from educosys_claude.observability.logger import get_logger

logger = get_logger(__name__)

def _session_file() -> Path:
    return Path(config["memory"]["db_path"]).parent / "current_session"

def get_current_session() -> str:
    session_file = _session_file()
    if session_file.exists():
        session_id = session_file.read_text().strip()
        logger.info(f"Resuming session: {session_id}")
        return session_id
    return new_session()

def new_session() -> str:
    session_id = str(uuid.uuid4())
    session_file = _session_file()
    session_file.parent.mkdir(exist_ok=True)
    session_file.write_text(session_id)
    logger.info(f"Started new session: {session_id}")
    return session_id

def switch_session(session_id: str) -> str:
    session_file = _session_file()
    session_file.write_text(session_id)
    logger.info(f"Switched to session: {session_id}")
    return session_id
```

memory/short_term.py

```
import sqlite3
from langgraph.checkpoint.sqlite import SqliteSaver
from pathlib import Path

from educosys_claude.config import config
from educosys_claude.observability.logger import get_logger

logger = get_logger(__name__)

def get_checkpointer() -> SqliteSaver:
    db_path = config["memory"]["db_path"]
    Path(db_path).parent.mkdir(exist_ok=True)
    logger.info(f"Using SQLite checkpointer at {db_path}")
    conn = sqlite3.connect(db_path, check_same_thread=False)
    return SqliteSaver(conn)
```

memory/__init__.py - new empty file

```
memory:
 db_path: .memory/memory.db
```

agent/factory.py

```
from langchain.agents import create_agent

from educosys_claude.llm.factory import get_llm
from educosys_claude.agent.tools import search_codebase
from educosys_claude.memory.short_term import get_checkpointer
from educosys_claude.observability.logger import get_logger

logger = get_logger(__name__)

SYSTEM_PROMPT = """You are a senior software engineer with deep knowledge of the codebase.
Always use the search_codebase tool before answering any question.
Reference specific file names, function names and line numbers in your answers.
If you cannot find the answer in the codebase, say so explicitly."""

def build_agent():
    """Create and return a LangChain agent with persistent memory."""
    llm = get_llm()
    tools = [search_codebase]
    checkpointer = get_checkpointer()

    return create_agent(
        llm,
        tools=tools,
        system_prompt=SYSTEM_PROMPT,
        checkpointer=checkpointer,
    )
```

agent/orchestrator.py

```
from educosys_claude.agent.factory import build_agent
from educosys_claude.observability.logger import get_logger

logger = get_logger(__name__)

def handle_query(question: str, thread_id: str) -> str:
    """Entry point for all user queries - builds the agent and runs it."""
    logger.info(f"Handling query for session {thread_id}: {question}")
    agent = build_agent()
    agent_config = {"configurable": {"thread_id": thread_id}}
    response = agent.invoke({"messages": [{"role": "user", "content": question}]},
    agent_config)
    return response["messages"][-1].content
```

main.py

```
import os
from pathlib import Path
from dotenv import load_dotenv
from rich.console import Console
from rich.prompt import Prompt

from educosys_claude.config import config
from educosys_claude.context.indexers.factory import get_indexer, get_index_inspector
from educosys_claude.llm.factory import get_llm, get_embedder
from educosys_claude.agent.orchestrator import handle_query
from educosys_claude.memory.session import get_current_session, new_session, switch_session
from educosys_claude.observability.logger import get_logger
# Load .env before anything else
load_dotenv(Path(__file__).parent.parent / ".env")

console = Console()
logger = get_logger(__name__)
```

```
def get_or_create_index():
    repo_path = str(Path.cwd())
    logger.info(f"Checking index for: {repo_path}")
    console.print(f"[dim]Checking index for {repo_path}...[/dim]")
    return get_indexer()(repo_path)

def initialize():
    """Bootstrap LLM, embedder, index and session before the REPL starts."""
    llm = get_llm()
    embedder = get_embedder()
    console.print(f"[dim]LLM: {config['llm']['provider']} / {config['llm']['model']}[/dim]")
    console.print(f"[dim]Embedder: {config['embeddings']['provider']} /
    {config['embeddings']['model']}[/dim]")

    index = get_or_create_index()
    session_id = get_current_session()
    console.print(f"[dim]Session: {session_id}[/dim]")
    console.print(f"[green]✓ Ready[/green]\n")
    return llm, embedder, index, session_id

def run():
    logger.info("Starting Educosys Claude")
    console.print("\n[bold blue]Educosys Claude[/bold blue] — RAG-powered code assistant")

    llm, embedder, index, session_id = initialize()
    console.print("Type [bold]'/exit'[/bold] to quit\n")

    while True:
        user_input = Prompt.ask("[bold green]>[/bold green]")

        if not user_input.strip():
            continue
        if user_input.lower() in ("/exit", "/quit"):
            logger.info("Shutting down")
            console.print("[dim]Goodbye![/dim]")
            break
        elif user_input.startswith("/ask "):
            question = user_input.removeprefix("/ask ").strip()
            logger.info(f"Ask command received: {question}")
            console.print(f"[dim]Searching for: {question}...[/dim]")
            response = handle_query(question, session_id)
            console.print(response)
        elif user_input == "/new_session":
            session_id = new_session()
            console.print(f"[green]New session started: {session_id}[/green]")
        elif user_input.startswith("/switch "):
            target = user_input.removeprefix("/switch ").strip()
            session_id = switch_session(target)
            console.print(f"[green]Switched to session: {session_id}[/green]")
        elif user_input == "/session":
            console.print(f"[dim]Current session: {session_id}[/dim]")
        elif user_input == "/show_index":
            logger.info("Showing index")
            get_index_inspector()(index)
        else:
            logger.warning(f"Unknown command received: {user_input}")
            console.print("[yellow]Unknown command. Try:[/yellow]")
            console.print("  [bold]/ask <question>[/bold]          — ask a question about the
    codebase")
            console.print("  [bold]/show_index[/bold]              — show all chunks in the
    index")
            console.print("  [bold]/new_session[/bold]             — start a fresh
    conversation")
            console.print("  [bold]/switch <session_id>[/bold]     — resume a past session")
            console.print("  [bold]/session[/bold]                 — show current session
    id")
if __name__ == "__main__":
    run()
```

Summarize STM

memory/short_term.py

```
import sqlite3
from langgraph.checkpoint.sqlite import SqliteSaver
from langchain.agents.middleware import SummarizationMiddleware
from pathlib import Path

from educosys_claude.config import config
from educosys_claude.llm.factory import get_llm
from educosys_claude.observability.logger import get_logger

logger = get_logger(__name__)


def get_checkpointer() -> SqliteSaver:
    db_path = config["memory"]["db_path"]
    Path(db_path).parent.mkdir(exist_ok=True)
    logger.info(f"Using SQLite checkpointer at {db_path}")
    conn = sqlite3.connect(db_path, check_same_thread=False)
    return SqliteSaver(conn)


def get_session_history(thread_id: str) -> list[dict]:
    checkpointer = get_checkpointer()
    config_ = {"configurable": {"thread_id": thread_id}}
    checkpoint = checkpointer.get(config_)
    if not checkpoint:
        return []
    messages = checkpoint["channel_values"].get("messages", [])
    return [
        {"role": "user" if m.type == "human" else "assistant", "content": m.content}
        for m in messages
    ]


def get_summarization_middleware() -> SummarizationMiddleware:
    return SummarizationMiddleware(
        model=get_llm(),
        trigger=("tokens", config["memory"]["summarize_at_tokens"]),
        keep=("messages", config["memory"]["keep_last_messages"]),
    )
```

agent/factory.py

```
from langchain.agents import create_agent
from educosys_claude.llm.factory import get_llm
from educosys_claude.agent.tools import search_codebase
from educosys_claude.memory.short_term import get_checkpointer, get_summarization_middleware
from educosys_claude.observability.logger import get_logger

logger = get_logger(__name__)

SYSTEM_PROMPT = """You are a senior software engineer with deep knowledge of the codebase.
Always use the search_codebase tool before answering any question.
Reference specific file names, function names and line numbers in your answers.
If you cannot find the answer in the codebase, say so explicitly."""

def build_agent():
    """Create and return a LangChain agent with persistent memory."""
    llm = get_llm()
    tools = [search_codebase]
    checkpointer = get_checkpointer()
```

```
    middleware = get_summarization_middleware()

    return create_agent(
        llm,
        tools=tools,
        system_prompt=SYSTEM_PROMPT,
        checkpointer=checkpointer,
        middleware=[middleware],
    )
```

config.yaml

```
memory:
 db_path: .memory/memory.db
 summarize_at_tokens: 4000
 keep_last_messages: 20
```

Tools

Terminal Tools

agent/tools/terminal_tools.py

```
import subprocess
import os
from langchain.tools import tool

_BLOCKED_COMMANDS = {"rm -rf /", "mkfs", "dd if=", ":(){:|:&};:"}
_TIMEOUT_SECONDS = 30

def _is_blocked(command: str) -> bool:
    return any(blocked in command for blocked in _BLOCKED_COMMANDS)

def _format_result(result: subprocess.CompletedProcess) -> str:
    parts = []
    if result.stdout:
        parts.append(result.stdout.rstrip())
    if result.stderr:
        parts.append(f"STDERR: {result.stderr.rstrip()}")
    if result.returncode != 0:
        parts.append(f"Exit code: {result.returncode}")
    return "\n".join(parts) if parts else "(no output)"

@tool
def run_command(command: str) -> str:
    """Run a shell command and return its output. Times out after 30 seconds."""
    if not command or not command.strip():
        return "Error: command cannot be empty"
    if _is_blocked(command):
        return "Error: command is not allowed for safety reasons"
    try:
        result = subprocess.run(
            command,
            shell=True,
            capture_output=True,
            text=True,
            timeout=_TIMEOUT_SECONDS,
        )
        return _format_result(result)
    except subprocess.TimeoutExpired:
        return f"Error: command timed out after {_TIMEOUT_SECONDS} seconds"
    except Exception as e:
        return f"Error: {e}"

@tool
def run_in_directory(command: str, directory: str) -> str:
    """Run a shell command inside a specific directory. Times out after 30 seconds."""
    if not command or not command.strip():
        return "Error: command cannot be empty"
    if not directory or not directory.strip():
        return "Error: directory cannot be empty"
    if not os.path.exists(directory):
        return f"Error: directory does not exist: {directory}"
    if not os.path.isdir(directory):
        return f"Error: path is not a directory: {directory}"
    if _is_blocked(command):
        return "Error: command is not allowed for safety reasons"
    try:
        result = subprocess.run(
            command,
            shell=True,
            capture_output=True,
```

```
            text=True,
            cwd=directory,
            timeout=_TIMEOUT_SECONDS,
        )
        return _format_result(result)
    except subprocess.TimeoutExpired:
        return f"Error: command timed out after {_TIMEOUT_SECONDS} seconds"
    except Exception as e:
        return f"Error: {e}"
```

Filesystem Tools

agent/tools/filesystem_tools.py

```
import os
from langchain.tools import tool

_MAX_FILE_SIZE_BYTES = 10 * 1024 * 1024  # 10 MB

@tool
def read_file(file_path: str) -> str:
    """Read and return the contents of a file."""
    if not file_path or not file_path.strip():
        return "Error: file path cannot be empty"
    if not os.path.exists(file_path):
        return f"Error: file not found: {file_path}"
    if not os.path.isfile(file_path):
        return f"Error: path is not a file: {file_path}"
    size = os.path.getsize(file_path)
    if size > _MAX_FILE_SIZE_BYTES:
        return f"Error: file too large ({size} bytes). Max allowed is {_MAX_FILE_SIZE_BYTES}
    bytes"
    try:
        with open(file_path, "r", encoding="utf-8") as f:
            return f.read()
    except UnicodeDecodeError:
        return f"Error: file is not valid UTF-8 text: {file_path}"
    except PermissionError:
        return f"Error: permission denied: {file_path}"
    except Exception as e:
        return f"Error: {e}"

@tool
def write_file(file_path: str, content: str) -> str:
    """Write content to a file, creating it and any parent directories if needed."""
    if not file_path or not file_path.strip():
        return "Error: file path cannot be empty"
    try:
        if os.path.dirname(file_path):
            os.makedirs(os.path.dirname(file_path), exist_ok=True)
        with open(file_path, "w", encoding="utf-8") as f:
            f.write(content)
        return f"Written to {file_path}"
    except PermissionError:
        return f"Error: permission denied: {file_path}"
    except Exception as e:
        return f"Error: {e}"


@tool
def append_file(file_path: str, content: str) -> str:
    """Append content to an existing file."""
    if not file_path or not file_path.strip():
        return "Error: file path cannot be empty"
```

```
    if not os.path.exists(file_path):
        return f"Error: file not found: {file_path}"
    if not os.path.isfile(file_path):
        return f"Error: path is not a file: {file_path}"
    try:
        with open(file_path, "a", encoding="utf-8") as f:
            f.write(content)
        return f"Appended to {file_path}"
    except PermissionError:
        return f"Error: permission denied: {file_path}"
    except Exception as e:
        return f"Error: {e}"

@tool
def delete_file(file_path: str) -> str:
    """Delete a file."""
    if not file_path or not file_path.strip():
        return "Error: file path cannot be empty"
    if not os.path.exists(file_path):
        return f"Error: file not found: {file_path}"
    if not os.path.isfile(file_path):
        return f"Error: path is not a file (use a directory tool for directories):
    {file_path}"
    try:
        os.remove(file_path)
        return f"Deleted {file_path}"
    except PermissionError:
        return f"Error: permission denied: {file_path}"
    except Exception as e:
        return f"Error: {e}"

@tool
def list_directory(directory: str) -> str:
    """List files and subdirectories inside a directory."""
    if not directory or not directory.strip():
        return "Error: directory cannot be empty"
    if not os.path.exists(directory):
        return f"Error: directory not found: {directory}"
    if not os.path.isdir(directory):
        return f"Error: path is not a directory: {directory}"
    try:
        entries = os.listdir(directory)
        return "\n".join(sorted(entries)) if entries else "(empty directory)"
    except PermissionError:
        return f"Error: permission denied: {directory}"
    except Exception as e:
        return f"Error: {e}"

@tool
def file_exists(file_path: str) -> str:
    """Check whether a file or directory exists."""
    if not file_path or not file_path.strip():
        return "Error: file path cannot be empty"
    return str(os.path.exists(file_path))
```

agent/factory.py

```
from langchain.agents import create_agent


from educosys_claude.llm.factory import get_llm
from educosys_claude.agent.tools import search_codebase
from educosys_claude.memory.short_term import get_checkpointer
from educosys_claude.observability.logger import get_logger
```

```
from educosys_claude.tools.terminal_tools import run_command, run_in_directory
from educosys_claude.tools.filesystem_tools import (
   read_file,
   write_file,
   append_file,
   delete_file,
   list_directory,
   file_exists,
)


logger = get_logger(__name__)


SYSTEM_PROMPT = """You are a senior software engineer with deep knowledge of the codebase.
Always use the search_codebase tool before answering any question.
Reference specific file names, function names and line numbers in your answers.
If you cannot find the answer in the codebase, say so explicitly."""


def build_agent():
  """Create and return a LangChain agent with persistent memory."""
  llm = get_llm()
  tools = [
      search_codebase,
      run_command,
      run_in_directory,
      read_file,
      write_file,
      append_file,
      delete_file,
      list_directory,
      file_exists,
  ]
  checkpointer = get_checkpointer()


  return create_agent(
      llm,
      tools=tools,
      system_prompt=SYSTEM_PROMPT,
      checkpointer=checkpointer,
  )
```

MCP

GitHub MCP

poetry add langchain_mcp_adapters

educosys_claude/educosys_mcp_servers.json

```
{
 "mcp_servers": {
   "github": {
     "command": "npx",
     "args": ["-y", "@modelcontextprotocol/server-github"],
     "env": {
       "GITHUB_PERSONAL_ACCESS_TOKEN": "${GITHUB_TOKEN}"
     },
     "transport": "stdio"
   }
 }
}
```

educosys_claude/mcp/__init__.py
educosys_claude/mcp/educosys_mcp_config.py

```
import os
import re
import json
from pathlib import Path
from dotenv import load_dotenv

load_dotenv()

_CONFIG_PATH = Path(__file__).parent.parent / "educosys_mcp_servers.json"

def load_educosys_mcp_configs() -> dict:
  """Return mcp_servers dict from educosys_mcp_servers.json with env vars resolved."""
  raw = json.loads(_CONFIG_PATH.read_text())
  # Replace ${VAR} placeholders in the config with actual env var values
  resolved = re.sub(r"\$\{(\w+)\}", lambda m: os.getenv(m.group(1), ""), json.dumps(raw))
  return json.loads(resolved).get("mcp_servers", {})
```

educosys_claude/mcp/educosys_mcp_client.py

```
from langchain_mcp_adapters.client import MultiServerMCPClient
from educosys_claude.mcp.educosys_mcp_config import load_educosys_mcp_configs
from educosys_claude.observability.logger import get_logger

logger = get_logger(__name__)

async def get_educosys_mcp_tools() -> list:
  """Connect to all configured MCP servers and return their tools."""
  configs = load_educosys_mcp_configs()
  logger.info(f"Connecting to MCP servers: {list(configs.keys())}")
  client = MultiServerMCPClient(configs)
  tools = await client.get_tools()
  logger.info(f"Loaded {len(tools)} tools from MCP servers")
  return tools
```

agent/factory.py

```
from langchain.agents import create_agent

from educosys_claude.llm.factory import get_llm
from educosys_claude.agent.tools import search_codebase
from educosys_claude.observability.logger import get_logger
from educosys_claude.tools.terminal_tools import run_command, run_in_directory
from educosys_claude.tools.filesystem_tools import (
   read_file,
   write_file,
   append_file,
   list_directory,
   file_exists,
)
from educosys_claude.mcp.educosys_mcp_client import get_educosys_mcp_tools

logger = get_logger(__name__)

SYSTEM_PROMPT = """You are a senior software engineer with deep knowledge of the codebase.
Always use the search_codebase tool before answering any question.
Reference specific file names, function names and line numbers in your answers.
If you cannot find the answer in the codebase, say so explicitly."""


async def build_agent(checkpointer):
   """Create and return a LangChain agent with persistent memory."""
   llm = get_llm()
   mcp_tools = await get_educosys_mcp_tools()
   tools = [
       search_codebase,
       run_command,
       run_in_directory,
       read_file,
       write_file,
       append_file,
       list_directory,
       file_exists,
       *mcp_tools,
   ]

   return create_agent(
       llm,
       tools=tools,
       system_prompt=SYSTEM_PROMPT,
       checkpointer=checkpointer,
   )
```

agent/orchestrator.py

```
from educosys_claude.observability.logger import get_logger

logger = get_logger(__name__)

async def handle_query(agent, question: str, thread_id: str) -> str:
  """Entry point for all user queries - invokes the agent."""
  logger.info(f"Handling query for session {thread_id}: {question}")
  agent_config = {"configurable": {"thread_id": thread_id}}
  try:
      response = await agent.ainvoke({"messages": [{"role": "user", "content": question}]},
   agent_config)
      return response["messages"][-1].content
  except Exception as e:
      logger.error(f"Agent error: {e}")
      return f"Error: {e}"
```

memory/short_term.py

```
from langgraph.checkpoint.sqlite.aio import AsyncSqliteSaver
from langchain.agents.middleware import SummarizationMiddleware
from pathlib import Path

from educosys_claude.config import config
from educosys_claude.llm.factory import get_llm
from educosys_claude.observability.logger import get_logger
logger = get_logger(__name__)

def get_checkpointer_db_path() -> str:
 db_path = config["memory"]["db_path"]
 Path(db_path).parent.mkdir(parents=True, exist_ok=True)
 logger.info(f"Using SQLite checkpointer at {db_path}")
 return db_path

def get_checkpointer() -> AsyncSqliteSaver:
 db_path = get_checkpointer_db_path()
 return AsyncSqliteSaver.from_conn_string(db_path)

def get_summarization_middleware() -> SummarizationMiddleware:
 return SummarizationMiddleware(
     model=get_llm(),
     trigger=("tokens", config["memory"]["summarize_at_tokens"]),
     keep=("messages", config["memory"]["keep_last_messages"]),
 )
```

main.py

```
import asyncio
from pathlib import Path
from dotenv import load_dotenv
from rich.console import Console
from rich.prompt import Prompt

from educosys_claude.config import config
from educosys_claude.context.indexers.factory import get_indexer, get_index_inspector
from educosys_claude.llm.factory import get_llm, get_embedder
from langgraph.checkpoint.sqlite.aio import AsyncSqliteSaver
from educosys_claude.agent.factory import build_agent
from educosys_claude.memory.short_term import get_checkpointer_db_path
from educosys_claude.agent.orchestrator import handle_query
from educosys_claude.memory.session import get_current_session, new_session, switch_session
from educosys_claude.observability.logger import get_logger

load_dotenv(Path(__file__).parent.parent / ".env")

console = Console()
logger = get_logger(__name__)


def get_or_create_index():
   repo_path = str(Path.cwd())
   logger.info(f"Checking index for: {repo_path}")
   console.print(f"[dim]Checking index for {repo_path}...[/dim]")
   return get_indexer()(repo_path)


async def initialize(checkpointer):
   """Bootstrap LLM, embedder, index, MCP tools, and session before the REPL starts."""
   llm = get_llm()
   embedder = get_embedder()
   console.print(f"[dim]LLM: {config['llm']['provider']} / {config['llm']['model']}[/dim]")
   console.print(f"[dim]Embedder: {config['embeddings']['provider']} /
   {config['embeddings']['model']}[/dim]")
```

```
    index = get_or_create_index()
    agent = await build_agent(checkpointer)
    session_id = get_current_session()
    console.print(f"[dim]Session: {session_id}[/dim]")
    console.print(f"[green]✓ Ready[/green]\n")
    return llm, embedder, index, agent, session_id


async def _run_async():
    logger.info("Starting Educosys Claude")
    console.print("\n[bold blue]Educosys Claude[/bold blue] — RAG-powered code assistant")

    async with AsyncSqliteSaver.from_conn_string(get_checkpointer_db_path()) as checkpointer:
        llm, embedder, index, agent, session_id = await initialize(checkpointer)
        console.print("Type [bold]'/exit'[/bold] to quit\n")

        while True:
            user_input = Prompt.ask("[bold green]>[/bold green]")

            if not user_input.strip():
                continue
            if user_input.lower() in ("/exit", "/quit"):
                logger.info("Shutting down")
                console.print("[dim]Goodbye![/dim]")
                break
            elif user_input.startswith("/ask "):
                question = user_input.removeprefix("/ask ").strip()
                logger.info(f"Ask command received: {question}")
                console.print(f"[dim]Searching for: {question}...[/dim]")
                response = await handle_query(agent, question, session_id)
                console.print(response)
            elif user_input == "/new_session":
                session_id = new_session()
                console.print(f"[green]New session started: {session_id}[/green]")
            elif user_input.startswith("/switch "):
                target = user_input.removeprefix("/switch ").strip()
                session_id = switch_session(target)
                console.print(f"[green]Switched to session: {session_id}[/green]")
            elif user_input == "/session":
                console.print(f"[dim]Current session: {session_id}[/dim]")
            elif user_input == "/show_index":
                logger.info("Showing index")
                get_index_inspector()(index)
            else:
                logger.warning(f"Unknown command received: {user_input}")
                console.print("[yellow]Unknown command. Try:[/yellow]")
                console.print("  [bold]/ask <question>[/bold]          — ask a question about
    the codebase")
                console.print("  [bold]/show_index[/bold]              — show all chunks in
    the index")
                console.print("  [bold]/new_session[/bold]             — start a fresh
    conversation")
                console.print("  [bold]/switch <session_id>[/bold]     — resume a past
    session")
                console.print("  [bold]/session[/bold]                 — show current session
    id")


def run():
    asyncio.run(_run_async())


if __name__ == "__main__":
    run()
```

Filesystem MCP

https://github.com/modelcontextprotocol/servers/tree/main/src/filesystem
educosys_claude/educosys_mcp_servers.json

```
{
 "mcp_servers": {
   "github": {
     "command": "npx",
     "args": ["-y", "@modelcontextprotocol/server-github"],
     "env": {
       "GITHUB_PERSONAL_ACCESS_TOKEN": "${GITHUB_TOKEN}"
     },
     "transport": "stdio"
   },
   "filesystem": {
       "command": "npx",
       "args": [
         "-y",
         "@modelcontextprotocol/server-filesystem",
         "${CWD}"
       ],
       "transport": "stdio"
     }
 }
}
```

Educosys_mcp_config.py - Replace CWD of project

```
import os
import re
import json
from pathlib import Path
from dotenv import load_dotenv

load_dotenv()

_CONFIG_PATH = Path(__file__).parent.parent / "educosys_mcp_servers.json"

def load_educosys_mcp_configs() -> dict:
 """Return mcp_servers dict from educosys_mcp_servers.json with env vars resolved.

 CWD is injected at load time so ${CWD} in the config always resolves to the
 directory where educosys claude was launched — not the package install location.
 """
 os.environ.setdefault("CWD", str(Path.cwd()))
 raw = json.loads(_CONFIG_PATH.read_text())
 # Replace ${VAR} placeholders in the config with actual env var values
 resolved = re.sub(r"\$\{(\w+)\}", lambda m: os.getenv(m.group(1), ""), json.dumps(raw))
 return json.loads(resolved).get("mcp_servers", {})
```

(Can remove Filesystem tools from agent/factory.py after adding this MCP Server)

Skills

https://platform.claude.com/docs/en/agents-and-tools/agent-skills/overview
https://docs.langchain.com/oss/python/deepagents/skills

https://github.com/anthropics/skills/blob/main/skills/frontend-design/SKILL.md
https://github.com/anthropics/skills/tree/main/skills/pdf

educosys_claude/skills/__init__.py

educosys_claude/skills/registry.py

```
from __future__ import annotations
import re
from pathlib import Path

from educosys_claude.observability.logger import get_logger
logger = get_logger(__name__)

# Every skill package must have this file at its root.
# Folders inside skills_dir that lack this file are skipped entirely.
SKILL_FILENAME = "SKILL.md"


class SkillNotFoundError(Exception):
    pass


class SkillRegistry:
    """
    Owns the in-memory catalog of all skills found in skills_dir.

    Expected layout on disk:
        skills_dir/
            python_debug/
                SKILL.md          ← required — frontmatter + instructions
                scripts/          ← optional — executable helpers
                templates/        ← optional — output templates
                resources/        ← optional — reference docs
            write_tests/
                SKILL.md
                templates/
                    pytest_template.py

    After calling load(), the internal _skills dict looks like:
        {
            "python_debug": {
                "meta":      {"name": "python_debug", "description": "...", ...},
                "body":      "You are an expert Python debugger. Follow these steps...",
                "skill_dir": Path(".educosys/skills/python_debug")
            },
            "write_tests": {
                "meta":      {"name": "write_tests", "description": "...", ...},
                "body":      "Generate pytest unit tests...",
                "skill_dir": Path(".educosys/skills/write_tests")
            }
        }

    "meta" is parsed from the YAML frontmatter block (name, description, when_to_use, ...).
    "body" is everything after the frontmatter — the actual instructions sent to the LLM.
    "skill_dir" is kept so we can later discover support files inside that folder.
    """
```

```python
    def __init__(self, skills_dir: Path) -> None:
        self._skills_dir = skills_dir
        self._skills: dict[str, dict] = {}  # populated by load()

    # -------------------------------------------------------------------
    # Startup: scan disk and populate _skills
    # -------------------------------------------------------------------

    def load(self) -> None:
        """
        Walk skills_dir and parse every SKILL.md found.
        Called once at agent startup via get_registry() in skill_tools.py.
        Safe to call again to hot-reload if skills change on disk.
        """
        self._skills.clear()

        if not self._skills_dir.exists():
            logger.warning(f"skills_dir not found: {self._skills_dir}. No skills loaded.")
            return

        for skill_dir in self._skills_dir.iterdir():
            if not skill_dir.is_dir():
                continue  # skip stray files at the top level of skills_dir

            skill_file = skill_dir / SKILL_FILENAME
            if not skill_file.exists():
                logger.warning(f"Skipping {skill_dir.name}: no SKILL.md found")
                continue

            try:
                meta, body = _parse_skill_file(skill_file)
                # Use "name" from frontmatter if declared; fall back to the folder name.
                # This means the folder name and the frontmatter name should match,
                # but the frontmatter value is what the agent will refer to.
                name = meta.get("name", skill_dir.name)
                self._skills[name] = {
                    "meta": meta,       # dict of frontmatter fields
                    "body": body,       # raw instruction text (sent to LLM on activation)
                    "skill_dir": skill_dir,  # kept for support file discovery
                }
                logger.info(f"Loaded skill: {name}")
            except Exception as e:
                logger.error(f"Failed to load skill from {skill_dir.name}: {e}")

    # -------------------------------------------------------------------
    # Called at agent build time → injected into SYSTEM_PROMPT
    # -------------------------------------------------------------------

    def build_skills_prompt(self) -> str:
        """
        Returns a compact string appended to the agent's SYSTEM_PROMPT at startup.

        ONLY includes metadata (name, description, when_to_use) — NOT the full
        skill body. This keeps the system prompt small. The agent loads full
        instructions on demand by calling the load_skill tool (progressive disclosure).

        Example output injected into SYSTEM_PROMPT:
            === Available Skills ===
            - python_debug: Debug Python errors and tracebacks | when_to_use: error,
    traceback, exception
            - write_tests: Generate pytest unit tests | when_to_use: test, coverage, pytest

            When the user's request matches a skill, call load_skill(name) ...
        """
        if not self._skills:
            return ""

        lines = ["=== Available Skills ==="]
```

```
        for name, skill in self._skills.items():
            meta = skill["meta"]
            description = meta.get("description", "No description provided.")
            when_to_use = meta.get("when_to_use", "")
            line = f"- {name}: {description}"
            if when_to_use:
                line += f" | when_to_use: {when_to_use}"
            lines.append(line)

        # This sentence is the agent's instruction on how to act when it recognises a skill.
        lines.append(
            "\nWhen the user's request matches a skill, call load_skill(name) "
            "to get the full instructions before proceeding."
        )
        return "\n".join(lines)

    # -------------------------------------------------------------------
    # Called at query time → triggered by the agent via the load_skill tool
    # -------------------------------------------------------------------

    def load_skill(self, name: str) -> str:
        """
        Returns the full SKILL.md body for the named skill, plus a listing of
        any support files inside the skill's folder.

        The agent calls this (via the load_skill @tool in skill_tools.py) after
        spotting a matching skill in the system prompt. The returned text becomes
        part of the agent's reasoning context for that query.

        Support files are listed but NOT read here — the agent decides which ones
        it needs and calls read_file() on them individually (tier-3 disclosure).

        Example return value:
            You are an expert Python debugger. Follow these steps...
            1. Ask for the full traceback if not provided.
            ...

            --- Support Files Available ---
              /path/to/skills/python_debug/scripts/run_debugger.py
              /path/to/skills/python_debug/resources/common_errors.md
            You can read any of these files using the read_file tool ...
        """
        if name not in self._skills:
            available = ", ".join(self._skills.keys()) or "none"
            raise SkillNotFoundError(
                f"Skill '{name}' not found. Available skills: {available}"
            )

        skill = self._skills[name]
        body = skill["body"]
        skill_dir: Path = skill["skill_dir"]

        # Discover support files and append their paths to the response.
        # The agent can then call read_file() on whichever ones are relevant.
        support_files = _list_support_files(skill_dir)
        result = body
        if support_files:
            result += "\n\n--- Support Files Available ---\n"
            result += "\n".join(f"  {p}" for p in support_files)
            result += (
                "\nYou can read any of these files using the read_file tool "
                "if the skill instructions reference them."
            )

        return result

    @property
    def skill_names(self) -> list[str]:
```

```
        return list(self._skills.keys())


# ---------------------------------------------------------------------------
# Module-level helpers (private to this file)
# ---------------------------------------------------------------------------

def _parse_skill_file(path: Path) -> tuple[dict, str]:
    """
    Read a SKILL.md and split it into (frontmatter_dict, body_str).

    A valid SKILL.md looks like:
        ---
        name: python_debug
        description: Debug Python errors and tracebacks
        when_to_use: error, traceback, exception
        ---
        You are an expert Python debugger...   ← this is the body

    The regex matches the opening ---, captures everything up to the closing ---,
    then captures the rest of the file as the body.
    If there is no frontmatter, the entire file is treated as the body and
    meta is returned as an empty dict (the folder name becomes the skill name).
    """
    text = path.read_text(encoding="utf-8")

    match = re.match(r"^---\s*\n(.*?)\n---\s*\n(.*)", text, re.DOTALL)
    if not match:
        # No frontmatter block found — body is the whole file.
        return {}, text.strip()

    frontmatter_str = match.group(1)  # raw YAML text between the --- markers
    body = match.group(2).strip()     # everything after the closing ---
    meta = _parse_yaml(frontmatter_str)
    return meta, body


def _parse_yaml(text: str) -> dict:
    """
    Parse the frontmatter YAML string into a dict.

    Strategy:
      1. Try yaml.safe_load (pyyaml is already a dependency via config.py).
      2. If yaml isn't importable for any reason, fall back to a line-by-line
         parser that handles flat "key: value" pairs only (no nesting or lists).
         This is enough for the required frontmatter fields.
    """
    try:
        import yaml  # already present — loaded by config.py at startup
        return yaml.safe_load(text) or {}
    except ImportError:
        pass

    # Minimal fallback for flat key: value frontmatter
    result = {}
    for line in text.splitlines():
        if ":" in line:
            key, _, value = line.partition(":")
            result[key.strip()] = value.strip()
    return result


def _list_support_files(skill_dir: Path) -> list[str]:
    """
    Return the absolute paths of every file inside the skill folder except SKILL.md.

    rglob("*") descends into all subdirectories (scripts/, templates/, resources/, ...),
    so the agent gets a complete picture of what resources the skill package ships with.
```

```
    Example for python_debug/:
        [
            "/path/to/skills/python_debug/scripts/run_debugger.py",
            "/path/to/skills/python_debug/resources/common_errors.md"
        ]
    """
    files = []
    for f in skill_dir.rglob("*"):
        if f.is_file() and f.name != SKILL_FILENAME:
            files.append(str(f))
    return files
```

educosys_claude/skills/skill_tools.py

```
from __future__ import annotations

from pathlib import Path

from langchain.tools import tool

from educosys_claude.config import config
from educosys_claude.skills.registry import SkillRegistry, SkillNotFoundError
from educosys_claude.observability.logger import get_logger

logger = get_logger(__name__)

# ---------------------------------------------------------------------------
# Singleton registry
# ---------------------------------------------------------------------------
# One SkillRegistry instance is shared for the entire process lifetime.
# It is created lazily on the first call to get_registry() — not at import time —
# so that Path.cwd() resolves to the target project directory, not the
# educosys_claude package directory.
#
# State transitions:
#   None  ──(first call to get_registry())──►  SkillRegistry (loaded)
#                                                      │
#                              all subsequent calls return this same instance
# ---------------------------------------------------------------------------
_registry: SkillRegistry | None = None


def get_registry() -> SkillRegistry:
    """
    Return the process-wide SkillRegistry, building it on first call.

    skills_dir resolution order:
      1. Read "skills.skills_dir" from config.yaml  →  e.g. ".educosys/skills"
      2. Resolve relative to Path.cwd()             →  the target project root
         (educosys_claude is invoked from the project being analysed, so cwd()
          IS the target project, not the educosys package itself)

    Example:
      User runs educosys_claude from /home/user/my_project/
      config.yaml says skills_dir: .educosys/skills
      → skills are loaded from /home/user/my_project/.educosys/skills/
    """
    global _registry
    if _registry is None:
        skills_dir = Path.cwd() / config.get("skills", {}).get("skills_dir",
    ".educosys/skills")
        _registry = SkillRegistry(skills_dir)
        _registry.load()
        logger.info(f"SkillRegistry initialized from: {skills_dir}")
    return _registry
```

```
# ---------------------------------------------------------------------------
# Called by factory.py at agent build time (not a tool — plain function)
# ---------------------------------------------------------------------------

def build_skills_prompt() -> str:
    """
    Trigger registry initialisation and return the metadata-only prompt snippet.

    This is called once inside build_agent() in factory.py and its return value
    is appended to SYSTEM_PROMPT before the agent is created. The LLM therefore
    always knows what skills exist, without paying the token cost of their bodies.

    Flow:
        factory.py: build_agent()
            → build_skills_prompt()          (this function)
                → get_registry()             (init registry if first call)
                    → SkillRegistry.load()   (scan skills_dir on disk)
                → SkillRegistry.build_skills_prompt()
                    → returns compact metadata string
            → appended to SYSTEM_PROMPT
    """
    return get_registry().build_skills_prompt()


# ---------------------------------------------------------------------------
# The agent-callable tool (registered in factory.py tools list)
# ---------------------------------------------------------------------------

@tool
def load_skill(name: str) -> str:
    """
    Load the full instructions for a skill by name.
    Call this when the user's request matches one of the skills listed in your system prompt.
    Returns the skill's step-by-step instructions and lists any available support files
    (scripts, templates, resources) that you can read using the read_file tool.
    """
    # The LLM calls this tool after spotting a matching skill name in its system prompt.
    # The tool result (full SKILL.md body + support file paths) becomes part of the
    # agent's reasoning context for this query only — it is NOT permanently added to
    # the system prompt or memory.
    #
    # Three-tier disclosure for a single query:
    #   Tier 1 — system prompt:  name + description (always present, ~5 tokens/skill)
    #   Tier 2 — this tool call: full SKILL.md body (loaded on demand, ~500-2000 tokens)
    #   Tier 3 — read_file():    individual support files (only if skill instructions say to)
    logger.info(f"Tool called: load_skill('{name}')")
    try:
        return get_registry().load_skill(name)
    except SkillNotFoundError as e:
        # Return the error as a plain string so the agent can recover gracefully
        # and tell the user, rather than raising an unhandled exception.
        return str(e)
```

educosys_claude/agent/factory.py

```
from langchain.agents import create_agent

from educosys_claude.llm.factory import get_llm
from educosys_claude.agent.tools import search_codebase
from educosys_claude.observability.logger import get_logger
from educosys_claude.tools.terminal_tools import run_command, run_in_directory
from educosys_claude.tools.filesystem_tools import (
  read_file,
  write_file,
```

```
  append_file,
  list_directory,
  file_exists,
)
from educosys_claude.mcp.educosys_mcp_client import get_educosys_mcp_tools
from educosys_claude.skills.skill_tools import load_skill, build_skills_prompt

logger = get_logger(__name__)

SYSTEM_PROMPT = """You are a senior software engineer with deep knowledge of the codebase.
Always use the search_codebase tool before answering any question.
Reference specific file names, function names and line numbers in your answers.
If you cannot find the answer in the codebase, say so explicitly."""

async def build_agent(checkpointer):
  """Create and return a LangChain agent with persistent memory."""
  llm = get_llm()
  mcp_tools = await get_educosys_mcp_tools()

  skills_prompt = build_skills_prompt()
  full_prompt = SYSTEM_PROMPT
  if skills_prompt:
      full_prompt = SYSTEM_PROMPT + "\n\n" + skills_prompt

  tools = [
      search_codebase,
      load_skill,
      run_command,
      run_in_directory,
      read_file,
      write_file,
      append_file,
      list_directory,
      file_exists,
      *mcp_tools,
  ]

  return create_agent(
      llm,
      tools=tools,
      system_prompt=full_prompt,
      checkpointer=checkpointer,
  )
```

config.yaml

```
skills:
 skills_dir: .educosys/skills
```

- To keep .chromadb and .memory in .educosys -

  Config.yaml for memory and chromadb -

```
skills:
 skills_dir: .educosys/skills

memory:
 db_path: .educosys/memory/memory.db
 summarize_at_tokens: 4000
 keep_last_messages: 20

chromadb:
 persist_dir: .educosys/chromadb/
 collection_name: codebase
```

Full updated config.yaml

```yaml
skills:
 skills_dir: .educosys/skills

memory:
 db_path: .educosys/memory/memory.db
 summarize_at_tokens: 4000
 keep_last_messages: 20

chromadb:
 persist_dir: .educosys/chromadb/
 collection_name: codebase

rag:
 mode: semantic           # semantic | hybrid

vector_store:
 provider: chroma
 retrieval_mode: dense  # dense | sparse | hybrid

vector_store:
 provider: chroma

qdrant:
 collection_name: educosys_claude

llm:
 provider: openai        # openai | anthropic
 model: gpt-5.5

embeddings:
 provider: openai
 model: text-embedding-3-small
```

Human in the Loop (HITL)

https://docs.langchain.com/oss/python/langchain/human-in-the-loop
requirements.txt

```
langchain
langchain-openai
python-dotenv
langgraph.checkpoint.sqlite
```

.env

```
OPENAI_API_KEY=...
SQLITE_CHECKPOINT_PATH=data/chatbot_memory.db
CHAT_THREAD_ID=demo-thread-1
```

```
import os
import json
import subprocess
from pathlib import Path

from dotenv import load_dotenv
from langchain.agents import create_agent
from langchain.agents.middleware import HumanInTheLoopMiddleware, ToolCallRequest
from langchain.chat_models import init_chat_model
from langchain.tools import tool
from langgraph.types import Command
from langgraph.checkpoint.sqlite import SqliteSaver


def _build_model():
    # Read the desired model from env; fall back to a small default model.
    model_name = os.getenv("OPENAI_MODEL", "").strip() or "gpt-5-nano"
    # init_chat_model understands "provider:model" strings, so default to
    # the openai provider when the user only supplied a bare model name.
    if ":" not in model_name:
        model_name = f"openai:{model_name}"
    return init_chat_model(model_name, temperature=0)


def _build_tools(workspace_root: Path) -> list:
    def _safe_resolve(user_path: str) -> Path:
        # Resolve the path and make sure it can't escape the workspace
        # (blocks things like "../../etc/passwd").
        candidate = (workspace_root / user_path).resolve()
        try:
            candidate.relative_to(workspace_root)
        except ValueError as exc:
            raise ValueError("Path must stay inside workspace root.") from exc
        return candidate

    @tool
    def list_files(path: str = ".") -> str:
        """List files and folders for a directory path within the workspace."""
        target = _safe_resolve(path)
        if not target.exists():
            return f"Path does not exist: {path}"
        if not target.is_dir():
            return f"Path is not a directory: {path}"

        # Sort directories first, then files, alphabetically (case-insensitive).
        entries = sorted(target.iterdir(), key=lambda p: (p.is_file(), p.name.lower()))
        if not entries:
```

```
        return f"Directory is empty: {path}"

    lines = []
    for entry in entries:
        rel = entry.relative_to(workspace_root).as_posix()
        kind = "dir" if entry.is_dir() else "file"
        lines.append(f"[{kind}] {rel}")
    return "\n".join(lines)

@tool
def read_file(path: str) -> str:
    """Read and return the full content of a UTF-8 text file."""
    target = _safe_resolve(path)
    if not target.exists():
        return f"File does not exist: {path}"
    if not target.is_file():
        return f"Path is not a file: {path}"
    return target.read_text(encoding="utf-8")

@tool
def write_file(path: str, content: str) -> str:
    """Create or overwrite a UTF-8 text file with provided content."""
    target = _safe_resolve(path)
    # Create any missing parent directories so writes never fail on that.
    target.parent.mkdir(parents=True, exist_ok=True)
    target.write_text(content, encoding="utf-8")
    rel = target.relative_to(workspace_root).as_posix()
    return f"Wrote file: {rel}"

@tool
def append_file(path: str, content: str) -> str:
    """Append content to a UTF-8 text file, creating it if missing."""
    target = _safe_resolve(path)
    target.parent.mkdir(parents=True, exist_ok=True)
    with target.open("a", encoding="utf-8") as handle:
        handle.write(content)
    rel = target.relative_to(workspace_root).as_posix()
    return f"Appended to file: {rel}"

@tool
def update_file(path: str, old_text: str, new_text: str, replace_all: bool = False) ->
str:
    """Update text in a file by replacing old_text with new_text."""
    target = _safe_resolve(path)
    if not target.exists():
        return f"File does not exist: {path}"
    if not target.is_file():
        return f"Path is not a file: {path}"

    content = target.read_text(encoding="utf-8")
    if old_text not in content:
        return "old_text was not found in file."

    if replace_all:
        updated = content.replace(old_text, new_text)
        count = content.count(old_text)
    else:
        # Only replace the first occurrence.
        updated = content.replace(old_text, new_text, 1)
        count = 1

    target.write_text(updated, encoding="utf-8")
    rel = target.relative_to(workspace_root).as_posix()
    return f"Updated file: {rel} (replacements: {count})"

@tool
def delete_file(path: str) -> str:
    """Delete a file at a given path within the workspace."""
```

```
        target = _safe_resolve(path)
        if not target.exists():
            return f"File does not exist: {path}"
        if not target.is_file():
            return f"Path is not a file: {path}"
        target.unlink()
        rel = target.relative_to(workspace_root).as_posix()
        return f"Deleted file: {rel}"

    @tool
    def run_terminal_command(command: str, timeout_seconds: int = 30) -> str:
        """Run a terminal command in workspace root and return output."""
        result = subprocess.run(
            command,
            cwd=workspace_root,
            shell=True,
            capture_output=True,
            text=True,
            # Clamp the timeout to a sane range (1s to 2 minutes).
            timeout=max(1, min(timeout_seconds, 120)),
        )
        stdout = result.stdout.strip()
        stderr = result.stderr.strip()
        parts = [f"exit_code: {result.returncode}"]
        if stdout:
            parts.append(f"stdout:\n{stdout}")
        if stderr:
            parts.append(f"stderr:\n{stderr}")
        return "\n\n".join(parts)

    @tool
    def ask_user(question: str) -> str:
        """Ask the human user for clarification or approval-related input."""
        # The real answer comes back later through the HITL interrupt/resume
        # flow; this return value is just a placeholder for the tool call.
        return f"User input placeholder for question: {question}"

    return [
        list_files,
        read_file,
        write_file,
        append_file,
        update_file,
        delete_file,
        run_terminal_command,
        ask_user,
    ]

def _is_dangerous_command(request: ToolCallRequest) -> bool:
    # Used as the "when" predicate for run_terminal_command's HITL rule:
    # only pause for approval if the command looks destructive.
    command = str(request.tool_call.get("args", {}).get("command", "")).lower()
    dangerous_tokens = [
        "rm ",
        "del ",
        "rmdir ",
        "format ",
        "shutdown",
        "reboot",
        "mkfs",
        "diskpart",
        "reg delete",
        "powershell -encodedcommand",
        "drop table",
        "truncate table",
    ]
    return any(token in command for token in dangerous_tokens)
```

```python
def _extract_state(result):
    # agent.invoke() can return either the raw state dict or an object
    # with a .value attribute depending on context; normalize to the dict.
    return getattr(result, "value", result)

def _print_agent_response(result) -> None:
    state = _extract_state(result)
    print(f"Bot: {state['messages'][-1].content}\n")

def _handle_interrupts(agent, result, config: dict) -> dict:
    # HumanInTheLoopMiddleware pauses the graph (an "interrupt") whenever a
    # gated tool call needs a human decision. Keep resolving interrupts and
    # resuming the agent until it finishes without asking for anything else.
    while getattr(result, "interrupts", None):
        interrupt = result.interrupts[0]
        payload = interrupt.value
        action_requests = payload.get("action_requests", [])
        review_configs = payload.get("review_configs", [])

        if not action_requests:
            break

        decisions = []
        for index, action in enumerate(action_requests):
            tool_name = action.get("name", "unknown_tool")
            arguments = action.get("args", {})
            allowed = review_configs[index].get(
                "allowed_decisions",
                ["approve", "edit", "reject", "respond"],
            )

            print(f"\nApproval needed for tool: {tool_name}")
            print(f"Arguments: {arguments}")
            print(f"Allowed decisions: {', '.join(allowed)}")

            # ask_user calls expect a free-text answer rather than an
            # approve/edit/reject decision.
            if "respond" in allowed and tool_name == "ask_user":
                user_reply = input("Your response: ").strip()
                decisions.append({"type": "respond", "message": user_reply})
                continue

            choice = input("Decision [approve/edit/reject]: ").strip().lower()
            while choice not in allowed:
                choice = input(f"Choose one of {allowed}: ").strip().lower()

            if choice == "approve":
                decisions.append({"type": "approve"})
            elif choice == "reject":
                feedback = input("Rejection feedback (optional): ").strip()
                decision = {"type": "reject"}
                if feedback:
                    decision["message"] = feedback
                decisions.append(decision)
            elif choice == "edit":
                print("Provide edited args as JSON object. Example:
    {\"path\":\"notes.txt\"}")
                edited_args_raw = input("Edited args JSON: ").strip()
                # Reuse the original arguments if the user submits nothing.
                edited_args = json.loads(edited_args_raw) if edited_args_raw else arguments
                decisions.append(
                    {
                        "type": "edit",
                        "edited_action": {"name": tool_name, "args": edited_args},
                    }
                )

        # Resume the graph from where it paused, supplying the human's
```

```
        # decisions for each pending tool call. This may trigger another
        # interrupt (loop continues) or return a final response.
        result = agent.invoke(
            Command(resume={"decisions": decisions}),
            config=config,
            version="v2",
        )
    return result

def main() -> None:
    # override=True: values in .env take precedence over already-set env vars.
    load_dotenv(override=True)

    if not os.getenv("OPENAI_API_KEY"):
        raise RuntimeError("OPENAI_API_KEY not found. Add it to .env before running.")

    model = _build_model()
    # Tools are sandboxed to the repo root (one level above this file).
    workspace_root = Path(__file__).resolve().parents[1]
    tools = _build_tools(workspace_root)

    db_path = os.getenv("SQLITE_CHECKPOINT_PATH", "data/chatbot_memory.db")
    # Make sure the checkpoint DB's parent directory exists before connecting.
    Path(db_path).parent.mkdir(parents=True, exist_ok=True)
    thread_id = os.getenv("CHAT_THREAD_ID", "demo-thread-1")

    print("Simple Agentic Chatbot (LangChain)")
    print("Type 'exit' to quit.\n")
    print(f"Thread ID: {thread_id}")
    print(f"Checkpoint DB: {db_path}\n")

    # SqliteSaver checkpoints conversation/agent state per thread_id, so the
    # chat can resume across process restarts.
    with SqliteSaver.from_conn_string(db_path) as checkpointer:
        # Gate specific tool calls behind human approval before they execute.
        hitl_middleware = HumanInTheLoopMiddleware(
            interrupt_on={
                "delete_file": {"allowed_decisions": ["approve", "reject"]},
                "write_file": {"allowed_decisions": ["approve", "edit", "reject"]},
                "append_file": {"allowed_decisions": ["approve", "edit", "reject"]},
                "update_file": {"allowed_decisions": ["approve", "edit", "reject"]},
                "run_terminal_command": {
                    "allowed_decisions": ["approve", "edit", "reject"],
                    # Only interrupt for commands that look destructive.
                    "when": _is_dangerous_command,
                },
                "ask_user": {"allowed_decisions": ["respond"]},
            },
            description_prefix="Tool execution pending approval",
        )
        agent = create_agent(
            model=model,
            tools=tools,
            system_prompt=(
                "You are a helpful demo assistant with file and terminal tools. "
                "Use tools when needed, explain actions briefly, and avoid destructive "
                "commands unless explicitly requested. If you need clarification from "
                "the human, call the ask_user tool."
            ),
            middleware=[hitl_middleware],
            checkpointer=checkpointer,
        )

        while True:
            user_input = input("You: ").strip()
            if not user_input:
                continue
            if user_input.lower() in {"exit", "quit"}:
```

```
                print("Bye!")
                break

            # thread_id ties this turn to the persisted conversation state.
            config = {"configurable": {"thread_id": thread_id}}
            result = agent.invoke(
                {"messages": [("user", user_input)]},
                config=config,
                version="v2",
            )
            # Resolve any HITL approval prompts before printing the reply.
            result = _handle_interrupts(agent, result, config)
            _print_agent_response(result)


if __name__ == "__main__":
    main()
```

Guardrails

https://docs.langchain.com/oss/python/langchain/guardrails

.env

```
GUARDRAIL_SHOW_REDACTED_INPUT=true
```

```
import os
from typing import Any

from dotenv import load_dotenv
from langchain.agents import create_agent
from langchain.agents.middleware import (
    AgentMiddleware,
    AgentState,
    PIIDetectionError,
    PIIMiddleware,
    hook_config,
)
from langchain.chat_models import init_chat_model
from langchain.messages import AIMessage
from langgraph.checkpoint.sqlite import SqliteSaver
from langgraph.runtime import Runtime

# Custom PII patterns (not covered by built-in email/credit_card/api_key detectors).
# Example matches for each pattern:
#   PASSWORD_SHARE_PATTERN:      "my password is Hunter2!", "pwd: amit123", "passwd=abc"
#   PASSWORD_IN_RESPONSE_PATTERN: "password: 'Hunter2!'", "pwd amit123" (matches the
#                                 model's own response text, not just user input)
#   API_KEY_SHARE_PATTERN:       "my api key is sk-proj-abc123", "apikey: xyz789"
PASSWORD_SHARE_PATTERN = (
    r"(?i)\b(?:my\s+)?(?:password|passwrod|passwd|pwd)\s*(?:is|=|:)\s*\S+"
)
PASSWORD_IN_RESPONSE_PATTERN = (
    r'(?i)(?:password|passwrod|passwd|pwd)\s*["\']?[\w!@#$%^&*.-]{3,}["\']?'
)
API_KEY_SHARE_PATTERN = r"(?i)\b(?:api[_\s-]?key|apikey)\s*(?:is|=|:)\s*\S+"


def _env_bool(name: str, default: bool = False) -> bool:
    # Parse a truthy env var string ("1"/"true"/"yes"/"on") into a bool.
    value = os.getenv(name, str(default)).strip().lower()
    return value in {"1", "true", "yes", "on"}

def _build_model():
    # Read the desired model from env; fall back to a small default model.
    raw_model_name = os.getenv("OPENAI_MODEL", "gpt-5-nano").strip()
    if not raw_model_name:
        raw_model_name = "gpt-5-nano"

    # Support "provider:model" syntax; otherwise assume the openai provider.
    if ":" in raw_model_name:
        provider, model_name = raw_model_name.split(":", 1)
        return init_chat_model(
            model=model_name,
            model_provider=provider,
            temperature=0,
        )

    return init_chat_model(
        model=raw_model_name,
        model_provider="openai",
        temperature=0,
    )
```

```python
class ContentFilterMiddleware(AgentMiddleware):
    """Deterministic guardrail: block banned keywords before the agent runs."""

    def __init__(self, banned_keywords: list[str]):
        super().__init__()
        self.banned_keywords = [keyword.lower() for keyword in banned_keywords]

    @hook_config(can_jump_to=["end"])
    def before_agent(self, state: AgentState, runtime: Runtime) -> dict[str, Any] | None:
        # Runs before the agent processes the turn; can short-circuit the
        # graph straight to "end" if the input is disallowed.
        if not state["messages"]:
            return None

        # Find the most recent human message to check for banned keywords.
        latest_user_message = None
        for message in reversed(state["messages"]):
            if getattr(message, "type", None) == "human":
                latest_user_message = message
                break

        if latest_user_message is None:
            return None

        content = str(latest_user_message.content).lower()
        for keyword in self.banned_keywords:
            if keyword in content:
                # Reject the request and skip straight to the end of the
                # graph instead of letting the model see the banned content.
                return {
                    "messages": [
                        {
                            "role": "assistant",
                            "content": (
                                "I cannot process requests containing inappropriate content.
    "
                                "Please rephrase your request."
                            ),
                        }
                    ],
                    "jump_to": "end",
                }

        return None


class RedactionDebugMiddleware(AgentMiddleware):
    """Print sanitized user input after PII middleware runs (demo only)."""

    def __init__(self, enabled: bool):
        super().__init__()
        self.enabled = enabled

    def before_model(self, state: AgentState, runtime: Runtime) -> dict[str, Any] | None:
        # Runs right before the model is called, i.e. after the PII
        # middlewares above it have already redacted/masked the input.
        if not self.enabled or not state["messages"]:
            return None

        for message in reversed(state["messages"]):
            if getattr(message, "type", None) == "human":
                print(f"[Guardrail Debug] Input sent to model: {message.content}")
                break

        return None
```

```python
class SafetyGuardrailMiddleware(AgentMiddleware):
    """Model-based guardrail: check final response safety after the agent completes."""

    def __init__(self, safety_model):
        super().__init__()
        self.safety_model = safety_model

    @hook_config(can_jump_to=["end"])
    def after_agent(self, state: AgentState, runtime: Runtime) -> dict[str, Any] | None:
        # Runs after the agent has produced its final answer, giving one
        # more chance to catch unsafe content (e.g. leaked credentials)
        # before it reaches the user.
        if not state["messages"]:
            return None

        last_message = state["messages"][-1]
        if not isinstance(last_message, AIMessage):
            return None

        # Ask a second model call to judge the safety of the final response.
        safety_prompt = (
            "Evaluate if this response is safe and appropriate for a demo assistant. "
            "Mark UNSAFE if the response repeats, quotes, or analyzes a user-provided "
            "password, secret, API key, or credential. Respond with only SAFE or UNSAFE.\n\n"
            f"Response: {last_message.content}"
        )
        result = self.safety_model.invoke([{"role": "user", "content": safety_prompt}])

        if "UNSAFE" in str(result.content).upper():
            # Overwrite the response in place rather than jumping/rerouting.
            last_message.content = (
                "I cannot provide that response. Please rephrase your request."
            )

        return None


def _build_guardrails(safety_model, show_redacted_input: bool) -> list:
    # Order matters: middleware runs in this sequence, so PII redaction
    # happens before the debug print and before the final safety check.
    middleware = [
        ContentFilterMiddleware(banned_keywords=["hack", "exploit", "malware",
    "ransomware"]),
        PIIMiddleware("email", strategy="redact", apply_to_input=True, apply_to_output=True),
        PIIMiddleware(
            "credit_card",
            strategy="mask",
            apply_to_input=True,
            apply_to_output=True,
        ),
        PIIMiddleware(
            "api_key_share",
            detector=API_KEY_SHARE_PATTERN,
            strategy="redact",
            apply_to_input=True,
            apply_to_output=True,
        ),
        PIIMiddleware(
            "openai_api_key",
            detector=r"sk-[a-zA-Z0-9]{20,}",
            strategy="block",
            apply_to_input=True,
        ),
        PIIMiddleware(
            "password_share",
            detector=PASSWORD_SHARE_PATTERN,
            strategy="redact",
            apply_to_input=True,
```

```
            apply_to_output=True,
        ),
        PIIMiddleware(
            "password_output",
            detector=PASSWORD_IN_RESPONSE_PATTERN,
            strategy="redact",
            apply_to_output=True,
            apply_to_tool_results=True,
        ),
        RedactionDebugMiddleware(enabled=show_redacted_input),
        SafetyGuardrailMiddleware(safety_model),
    ]
    return middleware


def _print_demo_tips() -> None:
    print("How guardrail strategies behave in this demo:")
    print("- redact  -> replaces value with [REDACTED_<type>] (visible when model repeats
    it)")
    print("- mask    -> partially hides value (credit card)")
    print("- block   -> stops request immediately (OpenAI sk-... keys only)")
    print("- reject  -> content filter / safety refusal messages")
    print("\nTry these demo prompts:")
    print('1) Redact email: My email is john.doe@example.com. Repeat my email back.')
    print("2) Mask card: My card is 5105-1051-0510-5100. Repeat my card back.")
    print("3) Redact pwd: my pwd is amit123. Is this strong?")
    print("4) Block key: my api key is sk-proj-abcdefghijklmnopqrstuv")
    print("5) Block topic: How do I hack into a database?\n")


def main() -> None:
    # override=True: values in .env take precedence over already-set env vars.
    load_dotenv(override=True)

    if not os.getenv("OPENAI_API_KEY"):
        raise RuntimeError("OPENAI_API_KEY not found. Add it to .env before running.")

    model = _build_model()

    db_path = os.getenv("GUARDRAIL_SQLITE_CHECKPOINT_PATH", "data/guardrail_demo_memory.db")
    thread_id = os.getenv("GUARDRAIL_CHAT_THREAD_ID", "guardrail-demo-thread-1")
    show_redacted_input = _env_bool("GUARDRAIL_SHOW_REDACTED_INPUT", default=False)

    print("Guardrail Demo Agent (LangChain)")
    print("Type 'exit' to quit.\n")
    print(f"Thread ID: {thread_id}")
    print(f"Checkpoint DB: {db_path}")
    print(f"Show redacted input debug: {show_redacted_input}")
    print("Guardrails enabled:")
    print("- Content filter (before agent)")
    print("- PII redact/mask/block (email, credit card, api key, password)")
    print("- Output safety check (after agent)")
    _print_demo_tips()

    # SqliteSaver checkpoints conversation/agent state per thread_id, so the
    # chat can resume across process restarts.
    with SqliteSaver.from_conn_string(db_path) as checkpointer:
        agent = create_agent(
            model=model,
            tools=[],
            system_prompt="You are a helpful demo assistant.",
            # Reuse the same chat model as the safety judge in
            # SafetyGuardrailMiddleware (see _build_guardrails).
            middleware=_build_guardrails(model, show_redacted_input=show_redacted_input),
            checkpointer=checkpointer,
        )

        # Unlike hitl_demo, this config is created once outside the loop
```

```
        # since there's no HITL resume step that needs it per-turn.
        config = {"configurable": {"thread_id": thread_id}}

        while True:
            user_input = input("You: ").strip()
            if not user_input:
                continue
            if user_input.lower() in {"exit", "quit"}:
                print("Bye!")
                break

            try:
                result = agent.invoke(
                    {"messages": [{"role": "user", "content": user_input}]},
                    config=config,
                )
            except PIIDetectionError as error:
                # Raised by the "block" strategy (openai_api_key detector)
                # instead of letting the message continue through the graph.
                print(
                    "Bot: Guardrail blocked your request because it contains a blocked "
                    f"OpenAI-style API key (sk-...).\nDetails: {error}\n"
                )
                continue

            print(f"Bot: {result['messages'][-1].content}\n")


if __name__ == "__main__":
    main()
```

Long-Running Tasks/Planning

main.py

```
from educosys_claude.tasks.orchestrator import handle_plan_command
from educosys_claude.tasks.status import show_task_status
```

```
          elif user_input.startswith("/plan "):
               goal = user_input.removeprefix("/plan ").strip()
               logger.info(f"Plan command received: {goal}")
               await handle_plan_command(goal)
          elif user_input == "/task_status":
               show_task_status()
```

```
              console.print("  [bold]/plan <goal>[/bold] — generate and execute a plan")
              console.print("  [bold]/task_status[/bold] — show task progress for active
   project")
```

config.yaml

```
tasks:
 db_path: .educosys/tasks.db

llm:
 provider: openai
 model: gpt-4o
 judge_model: gpt-4o-mini    # cheaper model used only for LLM-as-judge
```

Empty file - educosys_claude/tasks/__init__.py

Task Store

educosys_claude/tasks/task_store.py

```
from __future__ import annotations

import json
import sqlite3
import time
import uuid
from contextlib import contextmanager
from dataclasses import dataclass, field
from enum import Enum
from pathlib import Path

from educosys_claude.observability.logger import get_logger

logger = get_logger(__name__)


class TaskStatus(str, Enum):
   """
   All valid states a task can be in.

   State machine:
       PENDING ──(claim)──► IN_PROGRESS ──(success)──► COMPLETED  (terminal)
                                        │
                                        ├──(failure, retries left)──► PENDING
                                        ├──(failure, no retries)────► FAILED
                                        └──(dep failed)─────────────► BLOCKED
       SKIPPED  (terminal — human explicitly skipped)
```

```
    """
    PENDING     = "pending"
    IN_PROGRESS = "in_progress"
    COMPLETED   = "completed"
    FAILED      = "failed"
    BLOCKED     = "blocked"
    SKIPPED     = "skipped"

class TaskType(str, Enum):
    DESIGN    = "design"
    IMPLEMENT = "implement"
    TEST      = "test"
    REVIEW    = "review"
    INTEGRATE = "integrate"
    CONFIGURE = "configure"

@dataclass
class Task:
    """
    Single unit of work inside a project.

    depends_on and output_files are stored as JSON strings in SQLite
    and converted to/from lists at the Python boundary.
    """
    id: str
    project_id: str
    title: str
    description: str
    task_type: str
    status: str               = TaskStatus.PENDING
    depends_on: str           = "[]"   # JSON list of task IDs that must complete first
    output_files: str         = "[]"   # JSON list of file paths this task will produce
    result: str | None        = None   # agent's summary output on success
    error: str | None         = None   # error message on failure or crash
    retry_count: int          = 0
    max_retries: int          = 3
    execution_order: int      = 0      # position in plan — used for display ordering
    created_at: float         = field(default_factory=time.time)
    started_at: float | None  = None
    completed_at: float | None = None


class SQLiteTaskStore:
    """
    Authoritative persistence layer for all task state.

    Design rules:
    - WAL mode: crash-safe, allows concurrent reads while the orchestrator writes
    - Every connection is opened and closed per operation — no shared long-lived connection
    - Every state transition is a single atomic UPDATE — write to DB before acting on state
    - Execution is single-process and serial, so any task left IN_PROGRESS when the
      process starts up again is unconditionally orphaned (no heartbeat needed)

    DB layout:
        projects table  — one row per /plan invocation
        tasks table     — 10-40 rows per project, one per subtask
    """

    def __init__(self, db_path: str = ".educosys/tasks.db") -> None:
        self.db_path = db_path
        Path(db_path).parent.mkdir(parents=True, exist_ok=True)
        self._setup_db()
        logger.info(f"SQLiteTaskStore initialised at {db_path}")

    @contextmanager
    def _conn(self):
        """
        Open a fresh connection, enable WAL + foreign keys, yield, commit, close.
```

```
        Using a context manager per operation avoids shared-connection threading issues.
        """
        conn = sqlite3.connect(self.db_path, timeout=10)
        conn.execute("PRAGMA journal_mode=WAL")
        conn.execute("PRAGMA foreign_keys=ON")
        conn.row_factory = sqlite3.Row
        try:
            yield conn
            conn.commit()
        except Exception:
            conn.rollback()
            raise
        finally:
            conn.close()

    def _setup_db(self) -> None:
        with self._conn() as conn:
            conn.executescript("""
                CREATE TABLE IF NOT EXISTS projects (
                    id          TEXT PRIMARY KEY,
                    name        TEXT NOT NULL,
                    goal        TEXT NOT NULL,
                    plan_json   TEXT,
                    status      TEXT DEFAULT 'planning',
                    created_at  REAL DEFAULT (unixepoch()),
                    approved_at REAL
                );

                CREATE TABLE IF NOT EXISTS tasks (
                    id              TEXT PRIMARY KEY,
                    project_id      TEXT REFERENCES projects(id),
                    title           TEXT NOT NULL,
                    description     TEXT NOT NULL,
                    task_type       TEXT NOT NULL,
                    status          TEXT DEFAULT 'pending',
                    depends_on      TEXT DEFAULT '[]',
                    output_files    TEXT DEFAULT '[]',
                    result          TEXT,
                    error           TEXT,
                    retry_count     INTEGER DEFAULT 0,
                    max_retries     INTEGER DEFAULT 3,
                    execution_order INTEGER DEFAULT 0,
                    created_at      REAL DEFAULT (unixepoch()),
                    started_at      REAL,
                    completed_at    REAL
                );

                CREATE INDEX IF NOT EXISTS idx_tasks_project
                    ON tasks(project_id, status);
            """)

    # ------------------------------------------------------------------
    # Project operations
    # ------------------------------------------------------------------

    def create_project(self, goal: str, plan) -> str:
        """
        Persist an approved ExecutionPlan as a project + task rows.
        Returns the new project_id (UUID).
        """
        project_id = str(uuid.uuid4())
        with self._conn() as conn:
            conn.execute(
                "INSERT INTO projects(id, name, goal, plan_json, status) VALUES (?,?,?,?,?)",
                (project_id, plan.project_name, goal, plan.model_dump_json(), "approved")
            )
            for i, pt in enumerate(plan.tasks):
                conn.execute(
```

```
                """INSERT INTO tasks
                   (id, project_id, title, description, task_type,
                    depends_on, output_files, execution_order)
                   VALUES (?,?,?,?,?,?,?,?)""",
                (
                    pt.id, project_id, pt.title, pt.description,
                    pt.task_type.value,
                    json.dumps(pt.depends_on),
                    json.dumps(pt.output_files),
                    i,
                )
            )
    logger.info(f"Project {project_id} created with {len(plan.tasks)} tasks")
    return project_id

def get_latest_approved_project(self) -> str | None:
    """Return the most recently approved project_id, or None if none exists."""
    with self._conn() as conn:
        row = conn.execute(
            "SELECT id FROM projects WHERE status='approved' ORDER BY created_at DESC
LIMIT 1"
        ).fetchone()
    return row["id"] if row else None

# ------------------------------------------------------------------
# Atomic state transitions
# ------------------------------------------------------------------

def claim_task(self, task_id: str) -> bool:
    """
    Atomically transition a task from PENDING → IN_PROGRESS.
    Returns True if the claim succeeded, False if already claimed.
    """
    with self._conn() as conn:
        cur = conn.execute(
            """UPDATE tasks
               SET status     = 'in_progress',
                   started_at = unixepoch()
               WHERE id = ? AND status = 'pending'""",
            (task_id,)
        )
    return cur.rowcount == 1

def complete_task(self, task_id: str, result: str) -> None:
    with self._conn() as conn:
        conn.execute(
            """UPDATE tasks
               SET status       = 'completed',
                   result       = ?,
                   completed_at = unixepoch()
               WHERE id = ?""",
            (result, task_id)
        )
    logger.info(f"Task {task_id} completed")

def fail_task(self, task_id: str, error: str) -> None:
    """
    Record failure and apply retry logic in a single atomic UPDATE.
    SQL CASE: retries left → 'pending', exhausted → 'failed'.
    """
    with self._conn() as conn:
        conn.execute(
            """UPDATE tasks
               SET retry_count = retry_count + 1,
                   error       = ?,
                   status      = CASE
                       WHEN retry_count + 1 < max_retries THEN 'pending'
                       ELSE 'failed'
```

```
                    END,
                    started_at  = NULL
                WHERE id = ?""",
            (error, task_id)
        )
    logger.info(f"Task {task_id} failed: {error[:80]}")

def block_task(self, task_id: str, reason: str) -> None:
    with self._conn() as conn:
        conn.execute(
            "UPDATE tasks SET status='blocked', error=? WHERE id=?",
            (reason, task_id)
        )

# ------------------------------------------------------------------
# Queries
# ------------------------------------------------------------------

def get_ready_tasks(self, project_id: str) -> list[dict]:
    """Return PENDING tasks whose every dependency is COMPLETED or SKIPPED."""
    with self._conn() as conn:
        rows = conn.execute(
            """SELECT * FROM tasks
               WHERE project_id = ? AND status = 'pending'
               ORDER BY execution_order""",
            (project_id,)
        ).fetchall()

    ready = []
    for row in rows:
        deps = json.loads(row["depends_on"])
        if self._all_deps_done(project_id, deps):
            ready.append(dict(row))
    return ready

def _all_deps_done(self, project_id: str, dep_ids: list[str]) -> bool:
    if not dep_ids:
        return True
    placeholders = ",".join(["?"] * len(dep_ids))
    with self._conn() as conn:
        row = conn.execute(
            f"""SELECT COUNT(*) FROM tasks
                WHERE id IN ({placeholders})
                  AND status NOT IN ('completed', 'skipped')""",
            dep_ids
        ).fetchone()
    return row[0] == 0

def get_all_tasks(self, project_id: str) -> list[dict]:
    with self._conn() as conn:
        rows = conn.execute(
            "SELECT * FROM tasks WHERE project_id=? ORDER BY execution_order",
            (project_id,)
        ).fetchall()
    return [dict(r) for r in rows]

def get_progress(self, project_id: str) -> dict[str, int]:
    with self._conn() as conn:
        rows = conn.execute(
            "SELECT status, COUNT(*) as n FROM tasks WHERE project_id=? GROUP BY status",
            (project_id,)
        ).fetchall()
    return {r["status"]: r["n"] for r in rows}

def get_dep_results(self, dep_ids: list[str]) -> list[dict]:
    """
    Fetch title + result of completed dependency tasks.
    Injected into each agent's system prompt so it knows what prior tasks produced.
```

```
        """
        if not dep_ids:
            return []
        placeholders = ",".join(["?"] * len(dep_ids))
        with self._conn() as conn:
            rows = conn.execute(
                f"""SELECT id, title, result FROM tasks
                    WHERE id IN ({placeholders})
                      AND status = 'completed'""",
                dep_ids,
            ).fetchall()
        return [dict(r) for r in rows]
```

Planner

educosys_claude/tasks/planner.py

```
from __future__ import annotations

from pydantic import BaseModel
from langchain.agents import create_agent
from langchain.chat_models import init_chat_model

from educosys_claude.config import config
from educosys_claude.tasks.task_store import TaskType
from educosys_claude.observability.logger import get_logger

logger = get_logger(__name__)


class PlannedTask(BaseModel):
    id: str                          # stable snake_case e.g. task_001
    title: str
    description: str
    task_type: TaskType
    depends_on: list[str]            # list of task IDs that must complete first
    estimated_minutes: int
    output_files: list[str]          # files this task will create/modify
    acceptance_criteria: list[str]   # what "done" looks like for the judge


class ExecutionPlan(BaseModel):
    project_name: str
    goal_summary: str
    tech_stack: list[str]
    total_estimated_hours: float
    tasks: list[PlannedTask]
    risks: list[str]
    assumptions: list[str]


_SYSTEM_PROMPT = """\
You are a senior software architect. Given a software goal, produce a detailed
ExecutionPlan broken into concrete tasks for an AI agent to implement.

Rules:
- 5 to 20 tasks
- Task IDs must be stable snake_case: task_001, task_002, ...
- depends_on must reference valid task IDs in the same plan
- Ordering must form a valid DAG (no cycles): architecture → schema → config → core → tests
  → integration
- output_files must list every file the task will write to disk
- acceptance_criteria must be concrete and verifiable (3-5 items per task)
- task_type must be one of: design, implement, test, review, integrate, configure
```

```
"""


def create_plan(goal: str, extra_context: str = "") -> ExecutionPlan:
    """Call the LLM planner and return a structured ExecutionPlan."""
    provider = config["llm"]["provider"]
    model    = config["llm"]["model"]

    llm = init_chat_model(f"{provider}:{model}", temperature=0)

    planner_agent = create_agent(
        llm,
        tools=[],
        system_prompt=_SYSTEM_PROMPT,
        response_format=ExecutionPlan,
    )

    user_message = f"Goal: {goal}"
    if extra_context:
        user_message += f"\n\nAdditional context / change requests:\n{extra_context}"

    result = planner_agent.invoke({"messages": [{"role": "user", "content": user_message}]})
    plan: ExecutionPlan = result["structured_response"]
    logger.info(f"Plan created: {plan.project_name} with {len(plan.tasks)} tasks")
    return plan
```

Approval

educosys_claude/tasks/approval.py

```
from __future__ import annotations

from rich.console import Console
from rich.table import Table

from educosys_claude.tasks.planner import ExecutionPlan, PlannedTask

console = Console()


def present_plan_for_approval(plan: ExecutionPlan) -> ExecutionPlan | None:
    """
    Render the plan as a Rich table and prompt the user to:
      [A] Approve — returns the plan as-is
      [M] Modify  — edit a task description in-place and re-render
      [R] Reject  — returns None so the caller re-plans with feedback

    Loop continues until the user approves or rejects.
    """
    while True:
        _render_plan(plan)
        choice = input("\n[A]pprove / [M]odify task / [R]eject and re-plan:
    ").strip().upper()

        if choice == "A":
            return plan

        elif choice == "M":
            task_id = input("Enter task ID to modify: ").strip()
            task = next((t for t in plan.tasks if t.id == task_id), None)
            if not task:
                console.print(f"[red]Task '{task_id}' not found.[/red]")
                continue
            console.print(f"\nCurrent description:\n{task.description}\n")
```

```
            new_desc = input("New description: ").strip()
            if new_desc:
                task.description = new_desc
            console.print("[green]Task updated.[/green]")

        elif choice == "R":
            return None

        else:
            console.print("[yellow]Please enter A, M, or R.[/yellow]")


def _render_plan(plan: ExecutionPlan) -> None:
    console.print(f"\n[bold blue]Plan: {plan.project_name}[/bold blue]")
    console.print(f"[dim]{plan.goal_summary}[/dim]")
    console.print(f"[dim]Stack: {', '.join(plan.tech_stack)} | Est:
    {plan.total_estimated_hours}h[/dim]\n")

    table = Table(show_header=True, header_style="bold")
    table.add_column("ID",          style="dim",    width=12)
    table.add_column("Type",        width=10)
    table.add_column("Title",       width=32)
    table.add_column("Depends on",  width=20)
    table.add_column("Output files",width=30)

    for task in plan.tasks:
        table.add_row(
            task.id,
            task.task_type.value,
            task.title,
            ", ".join(task.depends_on) or "—",
            "\n".join(task.output_files) or "—",
        )

    console.print(table)

    if plan.risks:
        console.print("\n[yellow]Risks:[/yellow]")
        for r in plan.risks:
            console.print(f"  • {r}")
```

Executor

educosys_claude/tasks/executor.py

```
from __future__ import annotations

import json
from pydantic import BaseModel

from langchain.agents import create_agent
from langchain.chat_models import init_chat_model

from educosys_claude.config import config
from educosys_claude.tools.filesystem_tools import (
    append_file,
    file_exists,
    list_directory,
    read_file,
    write_file,
)
from educosys_claude.tools.terminal_tools import run_command
from educosys_claude.observability.logger import get_logger
```

```
logger = get_logger(__name__)


# Each task type gets a minimal, focused toolset — least privilege per task.
_TOOLS_BY_TYPE: dict[str, list] = {
    "design":    [read_file, write_file, list_directory],
    "implement": [read_file, write_file, append_file, list_directory],
    "test":      [read_file, write_file, append_file, list_directory, run_command],
    "review":    [read_file, write_file],
    "integrate": [read_file, write_file, append_file, list_directory, run_command],
    "configure": [read_file, write_file, list_directory, file_exists],
}

_DEFAULT_TOOLS = [read_file, write_file, list_directory]


def _parse_json_field(val) -> list:
    if isinstance(val, str):
        try:
            return json.loads(val)
        except Exception:
            return []
    return val or []


def _build_system_prompt(task: dict, dep_outputs: list[dict]) -> str:
    """
    Build a task-specific system prompt.

    dep_outputs — results from completed dependency tasks — are injected as
    "PRIOR TASK OUTPUTS" so the agent has memory of what was already built,
    without needing a shared checkpointer across tasks.
    """
    criteria_lines = "\n".join(
        f"  - {c}" for c in _parse_json_field(task.get("acceptance_criteria"))
    )
    output_files = "\n".join(
        f"  - {f}" for f in _parse_json_field(task.get("output_files"))
    )

    prior_context = ""
    if dep_outputs:
        parts = [
            f"[{dep['id']}] {dep['title']}\n{dep['result'] or '(no output recorded)'}"
            for dep in dep_outputs
        ]
        prior_context = "\n\nPRIOR TASK OUTPUTS (from your dependencies):\n" +
    "\n\n".join(parts)

    return f"""You are an expert software engineer executing a single well-defined task.
Be thorough and complete. Always write all files to disk before finishing.

TASK ID:    {task['id']}
TASK TYPE:  {task['task_type']}
TITLE:      {task['title']}

DESCRIPTION:
{task['description']}

FILES TO PRODUCE:
{output_files or '  (none specified)'}

ACCEPTANCE CRITERIA (your output must satisfy ALL of these):
{criteria_lines or '  (none specified)'}{prior_context}

When done, summarise what you implemented in 3-5 bullet points.
Do NOT leave any implementation incomplete."""
```

```python
# ---------------------------------------------------------------------
# LLM-as-judge
# ---------------------------------------------------------------------

class _JudgeVerdict(BaseModel):
    passed: bool
    score: int    # 0-10
    reason: str   # one sentence


_JUDGE_SYSTEM_PROMPT = """\
You are a code review judge. Given a task description, its acceptance criteria,
and the AI agent's output, decide whether the task was completed satisfactorily.

Score 0-10:
 8-10 → passed (all acceptance criteria met)
 5-7  → borderline (minor gaps, still usable)
 0-4  → failed (criteria not met, re-execution needed)

Set passed=true if score >= 6. One short sentence for reason."""


async def _judge_task(task: dict, agent_output: str) -> _JudgeVerdict:
    """
    Lightweight LLM-as-judge. Uses judge_model (cheaper) from config.
    Returns a structured verdict with passed/score/reason.
    """
    provider    = config["llm"]["provider"]
    judge_model = config.get("llm", {}).get("judge_model", config["llm"]["model"])

    llm = init_chat_model(f"{provider}:{judge_model}", temperature=0, max_tokens=1000)
    judge_agent = create_agent(
        llm,
        tools=[],
        system_prompt=_JUDGE_SYSTEM_PROMPT,
        response_format=_JudgeVerdict,
    )

    criteria = _parse_json_field(task.get("acceptance_criteria"))
    user_message = f"""TASK: {task['title']}
DESCRIPTION: {task['description']}
ACCEPTANCE CRITERIA: {json.dumps(criteria, indent=2)}

AGENT OUTPUT:
{agent_output[:2000]}"""

    result = await judge_agent.ainvoke({"messages": [{"role": "user", "content":
    user_message}]})
    return result["structured_response"]


# ---------------------------------------------------------------------
# Main entry point — called by orchestrator._execute()
# ---------------------------------------------------------------------

async def run_subtask_agent(task: dict, dep_outputs: list[dict] | None = None) -> str:
    """
    Build a fresh agent for a single task and invoke it.

    After the agent returns, an LLM judge verifies the output against
    acceptance_criteria. If it fails (score < 6), raises ValueError so
    the orchestrator's existing retry logic kicks in automatically.
    """
    provider = config["llm"]["provider"]
    model    = config["llm"]["model"]

    llm = init_chat_model(f"{provider}:{model}", temperature=0, max_tokens=3000)
```

```
    tools         = _TOOLS_BY_TYPE.get(task.get("task_type", ""), _DEFAULT_TOOLS)
    system_prompt = _build_system_prompt(task, dep_outputs or [])

    logger.info(f"Building agent for task {task['id']} (type={task['task_type']},
    tools={[t.name for t in tools]})")

    agent = create_agent(llm, tools=tools, system_prompt=system_prompt)

    # Be explicit that the project may be empty and files must be created from scratch.
    # Without this, the agent explores the empty directory and returns nothing.
    user_message = f"""{task['description']}

The project directory may be empty — there is no existing code to read.
You must CREATE all output files from scratch using write_file.
Do not spend time listing directories. Go directly to writing the output files."""

    final_state = None
    async for step in agent.astream(
        {"messages": [{"role": "user", "content": user_message}]},
        stream_mode="values",
    ):
        last_msg   = step["messages"][-1]
        kind       = type(last_msg).__name__
        tool_calls = getattr(last_msg, "tool_calls", None)
        if tool_calls:
            logger.info(f"Task {task['id']} → {kind} calling tools: {[tc['name'] for tc in
    tool_calls]}")
        else:
            preview = str(getattr(last_msg, "content", ""))[:200]
            logger.info(f"Task {task['id']} → {kind}: {preview}")
        final_state = step

    # Temporary debug — log full structure of last message to diagnose empty content
    last = final_state["messages"][-1]
    logger.info(f"DEBUG last message type: {type(last).__name__}")
    logger.info(f"DEBUG content: {repr(getattr(last, 'content', 'NO_CONTENT_FIELD'))}")
    logger.info(f"DEBUG tool_calls: {getattr(last, 'tool_calls', 'NO_TOOL_CALLS_FIELD')}")
    logger.info(f"DEBUG additional_kwargs: {getattr(last, 'additional_kwargs', {})}")
    logger.info(f"DEBUG response_metadata: {getattr(last, 'response_metadata', {})}")

    # Walk backwards through messages to find the last non-empty AIMessage content.
    # The final message can be an empty AIMessage if the model finishes a tool loop
    # without producing a text summary — this finds the actual last meaningful output.
    output = ""
    for msg in reversed(final_state["messages"]):
        if type(msg).__name__ == "AIMessage":
            content = getattr(msg, "content", "")
            # Claude returns content as a list of blocks; OpenAI returns a plain string.
            if isinstance(content, list):
                content = " ".join(
                    b.get("text", "") for b in content if isinstance(b, dict)
                )
            if content.strip():
                output = content
                break

    if not output.strip():
        raise ValueError("Agent returned empty output — it did not write any files or produce
    a summary")

    logger.info(f"Task {task['id']} agent returned {len(output)} chars")

    # ── LLM-as-judge ──────────────────────────────────────────
    verdict = await _judge_task(task, output)
    logger.info(
        f"Judge verdict for {task['id']}: score={verdict.score}, "
        f"passed={verdict.passed}, reason={verdict.reason}"
```

```
    )
    if not verdict.passed:
        raise ValueError(f"Judge rejected output (score={verdict.score}/10):
    {verdict.reason}")

    return output
```

Recovery

educosys_claude/tasks/recovery.py

```
from __future__ import annotations

from rich.console import Console

from educosys_claude.tasks.task_store import SQLiteTaskStore
from educosys_claude.observability.logger import get_logger

logger = get_logger(__name__)
console = Console()


class RecoveryManager:
    """
    Detects and resets tasks left IN_PROGRESS when the process crashed.

    In single-process serial execution, any IN_PROGRESS task at startup is
    unconditionally orphaned — the process that was running it is gone.
    No heartbeat or timing check is needed.
    """

    def __init__(self, store: SQLiteTaskStore) -> None:
        self.store = store

    def recover(self, project_id: str) -> int:
        """
        Reset every task still marked IN_PROGRESS for this project.
          - retries left  → reset to PENDING
          - no retries    → mark FAILED
        Returns number of tasks processed.
        """
        with self.store._conn() as conn:
            crashed = conn.execute(
                """SELECT id, title, retry_count, max_retries
                   FROM tasks
                   WHERE project_id = ? AND status = 'in_progress'""",
                (project_id,)
            ).fetchall()

            for task in crashed:
                if task["retry_count"] < task["max_retries"]:
                    conn.execute(
                        """UPDATE tasks
                           SET status      = 'pending',
                               started_at  = NULL,
                               retry_count = retry_count + 1,
                               error       = 'CRASH: process died mid-execution'
                           WHERE id = ?""",
                        (task["id"],)
                    )
                    console.print(
                        f"[yellow]🔄 Recovered:[/yellow] {task['id']} ({task['title']}) "
                        f"→ PENDING (retry {task['retry_count'] + 1}/{task['max_retries']})"
                    )
```

```
                else:
                    conn.execute(
                        """UPDATE tasks
                           SET status = 'failed',
                               error  = 'CRASH: max retries exceeded after repeated crashes'
                           WHERE id = ?""",
                        (task["id"],)
                    )
                    console.print(f"[red]❌ Max retries exhausted:[/red] {task['id']} →
    FAILED")

        count = len(crashed)
        if count:
            console.print(f"[dim]Recovery complete: {count} task(s) processed.[/dim]\n")
        return count
```

Orchestrator

educosys_claude/tasks/orchestrator.py

```
from __future__ import annotations

import asyncio
import json

from rich.console import Console

from pathlib import Path

from educosys_claude.config import config
from educosys_claude.tasks.task_store import SQLiteTaskStore
from educosys_claude.tasks.executor import run_subtask_agent
from educosys_claude.tasks.planner import create_plan
from educosys_claude.tasks.approval import present_plan_for_approval
from educosys_claude.tasks.recovery import RecoveryManager
from educosys_claude.context.indexers.factory import get_indexer
from educosys_claude.observability.logger import get_logger

logger = get_logger(__name__)
console = Console()


class TaskOrchestrator:
    """
    Main execution loop: finds tasks whose dependencies are met,
    claims them atomically, and dispatches them to subtask agents.

    Serial by default (max_concurrent=1) for cost control and determinism.
    """

    def __init__(self, store: SQLiteTaskStore, max_concurrent: int = 1) -> None:
        self.store = store
        self.max_concurrent = max_concurrent

    async def run(self, project_id: str) -> None:
        """
        Loop until all tasks reach a terminal state.

        Each iteration:
          1. Check progress — exit if nothing pending or in-progress
          2. Get ready tasks (all deps completed/skipped)
          3. Dispatch up to max_concurrent tasks
          4. Sleep 5s and repeat if waiting on in-progress tasks
        """
```

```
        console.print(f"\n[bold blue]Starting execution for project {project_id}[/bold
    blue]\n")

        while True:
            progress    = self.store.get_progress(project_id)
            pending     = progress.get("pending", 0)
            in_progress = progress.get("in_progress", 0)
            completed   = progress.get("completed", 0)
            failed      = progress.get("failed", 0)
            total       = sum(progress.values())

            console.print(
                f"[dim]Progress: {completed}/{total} completed"
                f" · {in_progress} in-progress"
                f" · {pending} pending"
                f" · {failed} failed[/dim]"
            )

            if pending == 0 and in_progress == 0:
                _print_final_summary(progress)
                break

            ready = self.store.get_ready_tasks(project_id)
            if not ready:
                if in_progress > 0:
                    console.print("[dim]⏳ Waiting for in-progress tasks...[/dim]")
                else:
                    console.print("[yellow]⚠ No tasks are ready — some may be blocked by
    failed dependencies.[/yellow]")
                    console.print("[yellow]  Use /task_status to inspect.[/yellow]")
                    break
                await asyncio.sleep(5)
                continue

            batch = ready[:self.max_concurrent]
            await asyncio.gather(*[self._execute(task) for task in batch])

    async def _execute(self, task: dict) -> None:
        """Claim and execute a single task, handling retries via fail_task."""
        console.print(f"\n[bold]▶ Starting:[/bold] [{task['id']}] {task['title']}")

        if not self.store.claim_task(task["id"]):
            console.print(f"[dim]⚠ Task {task['id']} already claimed — skipping[/dim]")
            return

        try:
            # Fetch what dependency tasks actually produced and inject into the agent.
            dep_ids     = json.loads(task.get("depends_on") or "[]")
            dep_outputs = self.store.get_dep_results(dep_ids)

            result = await run_subtask_agent(task, dep_outputs=dep_outputs)
            self.store.complete_task(task["id"], result)
            console.print(f"[green]✅ Completed:[/green] [{task['id']}] {task['title']}")

        except Exception as e:
            error_msg = str(e)
            self.store.fail_task(task["id"], error_msg)
            console.print(f"[red]❌ Failed:[/red]    [{task['id']}] {task['title']}:
    {error_msg[:120]}")
            logger.error(f"Task {task['id']} failed: {error_msg}")


async def handle_plan_command(goal: str) -> None:
    """
    Full /plan flow — entry point called by main.py.

      1. Check DB for an existing approved project → resume + recover if found
      2. Otherwise: plan → human approval loop → persist → execute
```

```
    """
    db_path = config.get("tasks", {}).get("db_path", ".educosys/tasks.db")
    store   = SQLiteTaskStore(db_path)
    recover = RecoveryManager(store)

    project_id = store.get_latest_approved_project()
    if project_id:
        console.print(f"\n[yellow]↩ Resuming existing project {project_id}...[/yellow]")
        recovered = recover.recover(project_id)
        if recovered:
            console.print(f"[dim]Recovered {recovered} crashed task(s)[/dim]")
    else:
        console.print("\n[dim]Planning with LLM (this may take a moment)...[/dim]")
        extra_context = ""
        approved_plan = None

        while approved_plan is None:
            raw_plan      = create_plan(goal, extra_context)
            approved_plan = present_plan_for_approval(raw_plan)
            if approved_plan is None:
                extra_context = input("What should change in the re-plan?\n> ").strip()
                console.print("\n[dim]Re-planning with your feedback...[/dim]")

        project_id = store.create_project(goal, approved_plan)
        console.print(f"\n[dim]Project {project_id} saved.[/dim]")

    orchestrator = TaskOrchestrator(store, max_concurrent=1)
    await orchestrator.run(project_id)

    # Re-index the project directory so /ask follow-up questions can find
    # the files that were just generated by the plan tasks.
    console.print("\n[dim]Re-indexing generated files so /ask can query them...[/dim]")
    try:
        get_indexer()(str(Path.cwd()))
        console.print("[green]✓ Index updated — you can now use /ask to ask about the
    generated code.[/green]")
    except Exception as e:
        logger.warning(f"Re-index after /plan failed: {e}")
        console.print(f"[yellow]⚠ Re-index failed: {e}[/yellow]")


def _print_final_summary(progress: dict[str, int]) -> None:
    completed = progress.get("completed", 0)
    failed    = progress.get("failed", 0)
    blocked   = progress.get("blocked", 0)
    skipped   = progress.get("skipped", 0)

    if failed == 0 and blocked == 0:
        console.print(f"\n[bold green]🎉 All {completed} tasks completed successfully![/bold
    green]")
    else:
        console.print(f"\n[bold yellow]⚠ Execution finished with issues:[/bold yellow]")
        console.print(f"  ✅ Completed: {completed}")
        if failed:
            console.print(f"  ❌ Failed:    {failed}  (run /task_status to review)")
        if blocked:
            console.print(f"  🚫 Blocked:   {blocked}  (dependencies failed)")
        if skipped:
            console.print(f"  ⏭ Skipped:   {skipped}")
```

Status

educosys_claude/tasks/status.py

```python
from __future__ import annotations

from rich.console import Console
from rich.table import Table

from educosys_claude.config import config
from educosys_claude.tasks.task_store import SQLiteTaskStore

console = Console()

_STATUS_STYLE = {
    "completed":   "green",
    "failed":      "red",
    "in_progress": "yellow",
    "pending":     "dim",
    "blocked":     "red",
    "skipped":     "dim",
}


def show_task_status() -> None:
    """Print a Rich table of all tasks for the latest approved project."""
    db_path    = config.get("tasks", {}).get("db_path", ".educosys/tasks.db")
    store      = SQLiteTaskStore(db_path)
    project_id = store.get_latest_approved_project()

    if not project_id:
        console.print("[yellow]No active project found. Run /plan <goal> first.[/yellow]")
        return

    tasks    = store.get_all_tasks(project_id)
    progress = store.get_progress(project_id)
    total    = sum(progress.values())
    done     = progress.get("completed", 0)

    console.print(f"\n[bold]Project:[/bold] {project_id}")
    console.print(f"[dim]Progress: {done}/{total} completed[/dim]\n")

    table = Table(show_header=True, header_style="bold")
    table.add_column("#",       width=4)
    table.add_column("ID",      width=12)
    table.add_column("Type",    width=10)
    table.add_column("Title",   width=35)
    table.add_column("Status",  width=12)
    table.add_column("Retries", width=8)
    table.add_column("Error",   width=40)

    for i, task in enumerate(tasks, 1):
        style  = _STATUS_STYLE.get(task["status"], "")
        error  = (task.get("error") or "")[:60]
        table.add_row(
            str(i),
            task["id"],
            task["task_type"],
            task["title"],
            f"[{style}]{task['status']}[/{style}]",
            f"{task['retry_count']}/{task['max_retries']}",
            error,
        )

    console.print(table)
```